

Table of Contents

ABSTRACT	1
Chapter 1: Introduction.....	2
1.1 Background	2
1.2 Objectives:	4
1.3 Scope:.....	5
1.4 Significance of the study:	6
Chapter 2: System Analysis and Design	8
2.1 User Requirement.....	8
2.2 User Interface	10
2.3 Integration Requirement	13
2.4 Data Design.....	11
2.5 ER Diagram	15
2.6 Data Flow.....	16
2.7 Project Scheduling.....	17
Gantt Chart.....	17
2.8 Hardware Requirement.....	21
2.9 Software Requirement	21
2.10 Module Division.....	22
Chapter 3: Data Dictionary	24
3.1 Data Dictionary	24
3.2 Purpose of Data Dictionary.....	26
3.3 Core Entities in the Data Dictionary	27
3.4 Data Relationships	28
3.5 Reflection as a Student	29
Chapter 4: Implementation & Testing.....	31
Testing.....	31
4.1 Unit Testing.....	31
4.2 Integration Testing.....	32

4.3 Beta Testing	33
4.4 Code Description	34
4.5 Code (Sample Implementation)	36
4.6 Screenshot.....	44
Chapter 5: System Architecture.....	54
5.1 Frontend Architecture.....	54
5.2 Backend Architecture.....	57
5.3 Database Architecture.....	60
Chapter 6: Results & Discussion	62
6.1 Introduction.....	62
6.2 Results Achieved	62
6.3 Discussion.....	64
6.4 Reflects	64
6.5 Key Observation.....	65
Chapter 7: Conclusion & Future Scope	66
7.1 Conclusion	66
7.2 Achievements.....	66
7.3 Limitations.....	67
7.4 Future Scope.....	68
Chapter 8: Bibliography	69

ABSTRACT

The rapid urbanization and growth of high-rise residential complexes in India have highlighted the increasing demand for organized, transparent, and technology-driven solutions for community living. **The Society Management System (SMS)** was conceptualized and developed as a comprehensive digital platform designed to streamline the end-to-end administrative operations of residential societies while enhancing communication between management and residents. This system is not merely a record-keeping tool but an integrated portal that combines inventory management, financial tracking, visitor security, and grievance redressal into a unified interface.

At its core, the portal provides administrators with seamless control over the society's physical and human assets. The system dynamically handles **Flats Management** and **Resident Allotments**, allowing for the digital mapping of blocks, floors, and apartment types. An enhanced **Billing Engine** allows administrators to generate maintenance bills based on specific journey details, such as flat size and monthly cycles, while providing residents with a transparent view of their financial obligations, payment history, and outstanding dues, thereby simplifying the complex process of manual ledger maintenance.

Beyond administrative logistics, the portal emphasizes resident safety and convenience. The **Visitor Management** module provides a digital security log, documenting guest entries, exits, and purpose of visit in real-time, replacing traditional paper registers. Furthermore, the **Complaint Management System** serves as a direct communication bridge; residents can lodge digital tickets for maintenance or security issues, which the management can then monitor, comment on, and resolve through a structured workflow.

A key technical highlight of the portal is its robust backend architecture. Built using **PHP** and **MySQL**, the system ensures data integrity through relational database design and secure session management. The integration of **Data Tables** allows for efficient server-side processing and instant searching of thousands of records. Security is a foundational element, with the implementation of **Bcrypt password hashing** to protect user credentials and role-based access control to ensure that sensitive financial and administrative data are accessible only to authorized personnel.

The portal's architecture combines **HTML5** and **CSS3** for a responsive and intuitive interface design, utilizing the **Bootstrap** framework to ensure accessibility across desktops and mobile devices. JavaScript-based modular event handling further enhances usability by providing dynamic interactions without full page reloads.

Ultimately, the **Society Management System** represents a holistic framework that bridges administrative efficiency, financial transparency, and resident security. It not only empowers committee members with tools to manage the society effectively but also fosters a professional and accountable environment for all inhabitants. Through its secure, interactive, and informative framework, the portal stands as a model of how modern web systems can transform the standard of living in urban residential sectors.

Chapter 1: Introduction

1.1 BACKGROUND:

The rapid pace of urbanization over the last few decades has fundamentally altered the way human beings live and interact within residential spaces. In metropolitan areas, the shift from independent houses to large-scale high-rise apartments and gated communities has become the new standard. These modern residential complexes are not merely a collection of individual homes; they function as complex micro-ecosystems, often housing hundreds or thousands of families within a single perimeter. Consequently, the administration of such large-scale communities has evolved into a significant logistical challenge, requiring a level of organization and efficiency that far exceeds the capabilities of traditional, manual methods.

Historically, the management of housing societies was a localized and informal affair. A small managing committee, often composed of volunteer residents, would handle the society's affairs using physical registers, paper-based accounting, and manual notice boards. For a long time, this system was sufficient. However, as the density of urban living increased, the limitations of manual management became glaringly apparent. In a manual system, information is fragmented. Financial records for maintenance fees and utility bills are often kept in separate ledgers, making it nearly impossible for residents to track their payment history or for the committee to identify defaulters in real-time. This lack of centralized data often leads to transparency issues, fostering a sense of mistrust between the residents and the administration.

Beyond finances, the security of these communities is a paramount concern. In the traditional setup, a visitor's entry is recorded by a security guard in a physical logbook. These logs are notoriously unreliable; handwriting is often illegible, entries can be easily falsified, and the data is practically impossible to search or analyze in the event of a security breach. Furthermore, the communication gap in a manual system is a constant source of friction. Important announcements are often posted on physical boards that many residents may never see, and the process of lodging a complaint such as reporting a plumbing issue or a broken elevator involves finding a committee member or writing in a suggestion book that may not be checked for days.

The emergence of the Digital Age has provided a solution to these systemic inefficiencies. The transition toward "Smart Cities" begins at the foundational level of the "Smart Society." A Society Management System (SMS) is a comprehensive web-based application designed to automate and integrate the diverse functions of a residential complex. By leveraging modern web technologies, specifically the WAMP/XAMPP stack (Windows, Apache, MySQL, and PHP), societies can move away from reactive, paper-based administration toward a proactive, data-driven management model.

The core philosophy behind this project is the centralization of information. Using MySQL as the backend database, the system creates a "Single Source of Truth." This means that every piece of data whether it is a resident's contact information, a record of a maintenance payment, or a visitor's entry time is stored in a structured, relational format. This architecture ensures

data integrity and allows for the generation of complex reports at the click of a button. For instance, the administration can instantly generate a list of all outstanding dues for a particular month, while a resident can log in to their private portal to view their specific billing history.

Furthermore, the use of PHP (Hypertext Preprocessor) as the server-side language allows for the creation of dynamic and interactive interfaces. Unlike static systems, a PHP-based SMS can handle real-time requests, such as updating the status of a resident's complaint from "Pending" to "In-Progress" or "Resolved." This level of responsiveness is crucial for maintaining resident satisfaction. When a resident knows that their grievance has been logged and is being tracked, it builds confidence in the management's competence.

Security is significantly bolstered through the digitization of visitor management. By replacing the physical logbook with a digital interface, the society can capture accurate visitor data, including phone numbers and the specific flat being visited. This data is not only searchable but also acts as a deterrent to unauthorized entry. In a modern context, where the safety of families and the security of property are top priorities, having a verifiable, digital trail of every individual who enters the premises is an invaluable asset.

The significance of this study also extends to the concept of the "Paperless Office." In an era where environmental sustainability is a global goal, reducing the reliance on paper for billing, notices, and record-keeping is a step in the right direction. Automation also eliminates the human error associated with repetitive tasks. Manual data entry is prone to mistakes that can lead to financial discrepancies or lost records. An automated system ensures that calculations for maintenance fees are precise and that notices are delivered digitally to every resident's account simultaneously.

Technologically, this project serves as a practical implementation of the Model-View-Controller (MVC) design pattern, albeit in a simplified form suitable for society needs. The HTML and CSS (using Bootstrap) handle the "View," ensuring the portal is accessible and easy to navigate for users of all ages. The PHP logic serves as the "Controller," processing user inputs and managing the flow of data, while the MySQL database acts as the "Model," holding the persistent data.

In conclusion, the development of a Society Management System is no longer a luxury but a necessity for modern urban living. As residential complexes continue to grow in size and complexity, the traditional ways of managing them are becoming obsolete. This study explores the design and implementation of a digital solution that addresses the core pillars of community management: financial transparency, security, communication, and administrative efficiency. By embracing these digital tools, housing societies can provide a safer, more organized, and more harmonious environment for all their members, truly reflecting the spirit of a digitally empowered society.

1.2 Objectives:

The primary objective of this study is to design, develop, and implement a robust **Society Management System (SMS)** that streamlines the administrative, financial, and security operations of a residential complex. As urban housing becomes increasingly complex, the reliance on fragmented, manual processes is no longer sustainable. This project aims to bridge the gap between traditional management and modern technological requirements by providing a unified platform for all stakeholders.

The specific objectives of this study are detailed below:

Centralization of Data and Administrative Automation:

The foremost goal is to eliminate the redundancy and errors associated with manual record-keeping. By utilizing a relational database management system, the project aims to consolidate resident profiles, flat details, and allotment history into a single, searchable repository. This ensures that the managing committee can access vital information instantly, reducing the time spent on clerical tasks and allowing for more focus on community development.

Enhancement of Financial Transparency:

A critical objective is the automation of the billing and payment lifecycle. The system is designed to generate maintenance bills periodically, track paid and outstanding amounts, and provide residents with digital receipts. By providing a clear, real-time view of the society's financial health, the system aims to reduce disputes between residents and the administration, ensuring that funds are collected and recorded with 100% accuracy.

Optimization of Security and Visitor Tracking:

In large residential complexes, security is often the weakest link due to the use of physical logbooks. This study aims to digitize the visitor management process. The objective is to create a module where security personnel can log every entry and exit digitally, capturing essential details such as the visitor's purpose and the host's flat number. This creates an immutable digital trail that can be audited in case of security incidents.

Improvement of Communication and Grief Redressal:

Effective communication is the backbone of any community. This project seeks to provide a dedicated **Complaints Management** module where residents can log issues (plumbing, electrical, etc.) directly through their portal. The objective is to move away from verbal or paper-based complaints to a system-driven approach where each grievance has a status (e.g., "Pending," "Resolved") and a timestamp, ensuring accountability for the maintenance staff and the committee.

Data-Driven Decision Making through Reporting:

The final objective is to empower the society's leadership with analytical tools. By implementing a **Reports Module**, the system will allow administrators to generate comprehensive summaries of financial collections, visitor frequency, and complaint resolution rates over specific periods. These insights are crucial for making informed decisions regarding budget allocations, security upgrades, and staff performance evaluations.

1.3 SCOPE:

The scope of the **Society Management System (SMS)** encompasses both the technical architecture of the platform and the practical functionalities designed to serve residents and administrators. Built using the **WAMP stack**, the system leverages **PHP** for robust server-side logic and **MySQL** for secure data persistence. The frontend is designed using **HTML, CSS, and Bootstrap**, ensuring that the interface is responsive across various devices, from desktop computers used in the society office to smartphones used by residents.

In terms of functional application, the scope includes the following modules:

User and Allotment Management:

Managing the complete lifecycle of a resident, from initial flat allotment and recording "move-in" dates to managing "move-out" transitions. It maintains a clear link between users and their specific residential units (blocks and flat numbers).

Financial & Billing Module:

Automating the generation of maintenance bills and tracking payment statuses. It includes a payment interface where residents can record transactions (e.g., via Cash), ensuring that the administration has a real-time record of paid versus outstanding amounts.

Visitor Management Module:

Replacing manual gate logs with a digital entry system. This module captures visitor names, contact details, the person they are meeting, and precise "In" and "Out" timestamps to maintain a secure digital trail of all movement within the premises.

Grievance Redressal (Complaints) Module:

Providing a structured channel for residents to report maintenance issues. This includes the ability to describe the problem, track its progress, and allow administrators to provide comments and update the status of the complaint from "Pending" to "Resolved."

Reporting & Analytics Module:

Equipping the management committee with the ability to generate date-specific reports. The scope includes filtered data exports for bills, complaints, and visitor logs, facilitating data-driven reviews of society operations.

Security & Authentication:

Implementing secure login protocols for both 'Admin' and 'User' roles, including password hashing and session management to ensure that sensitive data is only accessible to authorized personnel.

1.4 SIGNIFICANCE OF THE STUDY:

The significance of this study lies in its potential to modernize and standardize the management of residential communities. As urban populations shift toward high-rise apartments and gated communities, the traditional methods of manual oversight are proving to be inefficient, non-transparent, and insecure. This project provides a scalable digital solution that benefits various stakeholders within the ecosystem of a housing society.

For Society Administrators

The study is highly significant for the managing committee and administrative staff. By transitioning from physical registers to a **MySQL-based digital database**, administrators can significantly reduce the risk of data loss or manipulation. The automation of maintenance bill generation and payment tracking eliminates the tedious task of manual calculation, thereby reducing human error. It also provides a structured way to handle complaints, ensuring that no resident's concern is overlooked. This leads to higher operational efficiency and a more professional management approach.

For Residents

For the residents, the significance is rooted in convenience and transparency. The system empowers residents by providing them with a personal portal where they can view their billing history, track the status of their complaints, and receive community-wide notifications. The digital record of payments acts as a verified proof of transaction, reducing financial disputes with the management. Furthermore, the systematic handling of grievances ensures a better quality of life through timely maintenance and repairs.

For Security and Safety

One of the most vital aspects of this study is the improvement of community security. By implementing a digital **Visitor Management Module**, the study replaces unreliable paper logs with an immutable digital trail. This is significant for maintaining the safety of the premises, as security personnel can easily track the frequency of visitors and ensure that all guests are accounted for. In the event of an incident, the digital logs provide searchable evidence that is far more effective than scanning through months of handwritten books.

For the Field of Information Technology

From a technical perspective, this study serves as a practical demonstration of how the **WAMP (Windows, Apache, MySQL, PHP) stack** can be applied to solve real-world organizational problems. It highlights the importance of **CRUD (Create, Read, Update, Delete) operations** in maintaining data integrity and showcases the use of **Session Management** to secure sensitive user information. For students and developers, this study provides a blueprint for

building modular, database-driven web applications that can be adapted for other organizational management needs.

Long-term Strategic Value

Ultimately, the study contributes to the concept of "Smart Living." By creating a centralized repository of society data, the project enables long-term trend analysis. Administrators can identify recurring maintenance issues, seasonal visitor patterns, and financial trends. This data-driven approach allows for better budgeting and strategic planning, ensuring the long-term sustainability and value appreciation of the property.

Chapter 2: System Analysis & Design

2.1 USER REQUIREMENT:

The User Requirement section defines the specific needs and expectations of the stakeholders who will interact with the Society Management System (SMS). Based on the system's architecture, there are two primary user roles: the Administrator and the Resident (User). Each role requires a distinct set of functionalities to ensure efficient management and communication within the society.

2.1.1 Stakeholder Identification:

Administrator (Admin):

The administrative user responsible for the overall management of the society's data, financial records, and maintenance requests.

Resident (User):

The occupant of a flat who uses the system to manage personal bills, track visitors, and communicate issues to the management

2.1.2 Functional Requirements for Administrator

The Administrator requires comprehensive control over the system's data modules:

User Management:

The Admin must be able to add, edit, and remove resident accounts, including managing their names, email addresses, and secure login credentials.

Property Management:

The system must allow the Admin to maintain a record of all flats, including details such as flat number, floor, block number, and flat type.

Allotment Tracking:

The Admin requires the ability to assign residents to specific flats and track move-in/move-out dates.

Billing and Financials:

The system must enable the Admin to generate monthly bills for specific flats, define bill titles, and record the amounts due.

Visitor Management:

The Admin must have the capability to log visitor entries, including the visitor's name, phone number, address, the person they are meeting, and the reason for the visit.

Maintenance and Complaints:

The Admin must be able to view all submitted complaints from residents, update their status (e.g., pending or resolved), and provide master comments on the actions taken.

Reporting:

The Admin requires a reporting tool to generate summary data for bills, complaints, and visitor logs over specific date ranges.

2.1.3 Functional Requirements for Resident (User)

The Resident requires a streamlined interface focused on their specific unit:

Bill Management:

Residents must be able to view their pending and paid bills and initiate payment records within the system.

Complaint Submission:

The system must allow residents to submit maintenance or service complaints with a detailed description of the issue.

Visitor Monitoring:

Residents require access to view the logs of visitors who have visited their specific flat for security verification.

Profile Management:

Users must be able to update their personal information and change their account passwords to maintain security.

2.1.4 Non-Functional Requirements

To ensure the system is robust and professional, the following non-functional requirements must be met:

Security:

The system must use industry-standard hashing (e.g., password_hash) to protect user credentials.

Usability:

The interface should be responsive and intuitive, utilizing modern frameworks like Bootstrap to ensure compatibility across devices.

Data Integrity:

The database must enforce constraints (foreign keys) to ensure that records like bills and allotments are always linked to valid flats and users.

Performance:

The system should utilize server-side processing for large data tables to ensure fast loading times during search and filtering operations.

2.2 USER INTERFACE

The User Interface (UI) design for the Society Management System (SMS) focuses on providing a clean, intuitive, and responsive environment for both administrators and residents. The primary objective is to facilitate complex management tasks—such as financial tracking and visitor logging—through a simplified visual hierarchy. The system utilizes the **SB Admin** dashboard template, built on the **Bootstrap 5** framework, ensuring a "mobile-first" approach that allows users to access the system via desktops, tablets, or smartphones.

2.2.1 Design Principles and Layout

The interface is structured using a standard administrative layout, consisting of a top navigation bar and a persistent sidebar.

Top Navigation Bar:

Contains the system branding and quick-access tools, such as the sidebar toggle and user profile settings.

Sidebar Navigation:

Serves as the primary menu. For the Admin, it provides links to all management modules (Flats, Users, Allotments, Bills, Complaints, Visitors, and Reports). For the Resident, the sidebar is dynamically filtered to show only relevant personal modules (Bills, Complaints, and Visitors).

Breadcrumb Navigation:

Each page includes breadcrumb links below the header to help users track their location within the system hierarchy and navigate back to the dashboard easily.

2.2.2 Key Interface Components

The UI is designed around several recurring components to maintain consistency:

Data Tables:

The system heavily utilizes DataTables for listing records. These tables feature server-side processing, allowing users to search, sort, and filter through hundreds of records (e.g., visitor logs or billing history) without performance lag.

Actionable Cards:

The dashboard uses "Stat Cards" to provide an immediate overview. For example, the Admin dashboard displays cards for "Total Flats," "Total Bills," and "Pending Complaints," using color-coded badges to indicate status (e.g., Red for "In" visitors, Green for "Resolved" complaints).

Modal and Inline Forms:

To minimize page reloads, many interactions occur through structured forms. These forms utilize Bootstrap's validation states, providing immediate visual feedback (red borders for errors, green for success) when a user misses a required field like a flat number or bill amount.

2.2.3 Administrator Interfaces

The Admin UI is data-centric. The **Flats Management** and **User Management** screens provide a birds-eye view of the society's data. When adding or editing a record—such as a new bill—the interface uses dropdown menus populated with real-time database values (e.g., selecting a flat from a list of blocks) to prevent manual entry errors. The **Reports** interface is particularly robust, featuring date-picker inputs and dropdowns to generate specific data visualizations for complaints or financial audits.

2.2.4 Resident (User) Interfaces

The Resident interface is simplified to reduce cognitive load. Upon logging in, the resident is greeted with a dashboard showing their specific unit's status. The **Complaint Submission** screen is a simple text area where residents can describe issues, while the **Bill Payment** screen provides a clear table of pending dues with a "Payment" button that triggers a focused transaction form.

2.2.5 Visual Aesthetics

The visual identity of the SMS is professional and minimalist. It uses a palette of dark grays for navigation elements (bg-dark), white for content areas to maximize readability, and standard "Signal" colors for status updates (Blue for information, Yellow for warnings, Red for danger/pending, and Green for success/resolved). Typography is handled via the system-default sans-serif stack, ensuring high legibility across different operating systems. High-quality icons from the **FontAwesome 6** library are used throughout the sidebar and action buttons to provide visual cues for functionality.

2.3 INTEGRATION REQUIREMENT:

Based on the provided source code for the Society Management System (SMS), the **Integration Requirements** describe how the various software components, modules, and external libraries work together to form a functional system.

2.3.1 Database Integration

The system requires seamless integration between the PHP application and a MySQL database to manage persistent data.

PDO Connection:

The application uses the **PHP Data Objects (PDO)** extension to establish a secure and consistent connection to the database (DB_HOST, DB_NAME, DB_USER, DB_PASS).

Schema Synchronization:

The config.php file includes logic to automatically import the database.sql schema if the system detects that required tables (like users) do not exist, ensuring the database structure matches the application's expectations.

2.3.2 Internal Module Integration (Data Relationships)

The core modules (Users, Flats, Allotments, Bills, Complaints, and Visitors) are integrated at the database level through relational constraints:

Allotments Integration:

Connects the users and flats tables using foreign keys (user_id, flat_id), allowing the system to track which user resides in which flat.

Financial Integration:

The bills module is linked to the flats table via flat_id, ensuring every generated bill is associated with a specific property.

Issue Management Integration:

The complaints module integrates both users and flats, allowing administrators to see who raised a complaint and the specific unit affected.

Visitor Management Integration:

Links visitor records to the flats table to identify which unit a visitor is visiting.

2.3.3 Frontend-Backend Integration (AJAX/DataTables)

The system integrates server-side processing with the client-side user interface to handle large datasets efficiently:

Asynchronous Data Fetching:

Modules like Flats and Allotments use **jQuery DataTables** integrated with **AJAX**.

Centralized Request Handling:

The `action.php` file serves as a centralized controller that receives POST requests (e.g., action: 'fetch_flats'), queries the database, and returns JSON-formatted data to be rendered on the frontend.

2.3.4 UI and Framework Integration

The application integrates several third-party libraries to provide a responsive and professional user interface:

Bootstrap 5 Integration:

Used for the overall layout, responsive grid system, and UI components like alerts, cards, and forms.

SB Admin Template:

The project is integrated into the "SB Admin" dashboard template, which provides the sidebar navigation and layout structure.

FontAwesome:

Integrated via script tags in the header to provide iconography across the navigation and action buttons.

2.3.5 Authentication and Authorization Integration

Access control is integrated throughout the application using PHP sessions:

Session-Based Navigation:

The `header.php` file dynamically integrates different navigation links based on the `$_SESSION['user_role']` (Admin vs. User).

Role-Based Access Control (RBAC):

Individual pages (like `add_visitor.php` or `allotments.php`) integrate a check at the beginning of the file to verify if the user is logged in and has the appropriate administrative permissions before rendering content.

2.4 DATA DESIGN:

Data design outlines how the information within the system is organized, stored, and accessed. For the Society Management System, this includes structured representations of residents, physical apartment units, financial obligations, and maintenance issues. The system utilizes a relational MySQL database to maintain data integrity across these interconnected modules.

Data Entities

The primary entities in the system are:

1. User

Attributes: id, name, email, password, role (Admin/User)

Purpose: Represents the individuals authorized to access the system, distinguishing between administrative staff and society residents.

2. Flat

Attributes: id, flat_number, floor, block_number, flat_type (e.g., 1BHK, 2BHK), created_at

Purpose: Represents the physical unit or property within the society complex.

3. Allotment

Attributes: id, user_id, flat_id, move_in_date, move_out_date, created_at

Purpose: Acts as a junction entity that links Users to specific Flats, tracking the duration of residency.

4. Bill

Attributes: id, flat_id, bill_title, amount, month, paid_amount, paid_date, payment_method, status

Purpose: Manages the financial records for maintenance fees or utility charges associated with each flat.

5. Complaint

Attributes: id, user_id, flat_id, description, master_comment, status (Pending/Resolved), created_at

Purpose: Stores maintenance requests or grievances filed by residents and tracks the resolution status provided by the admin.

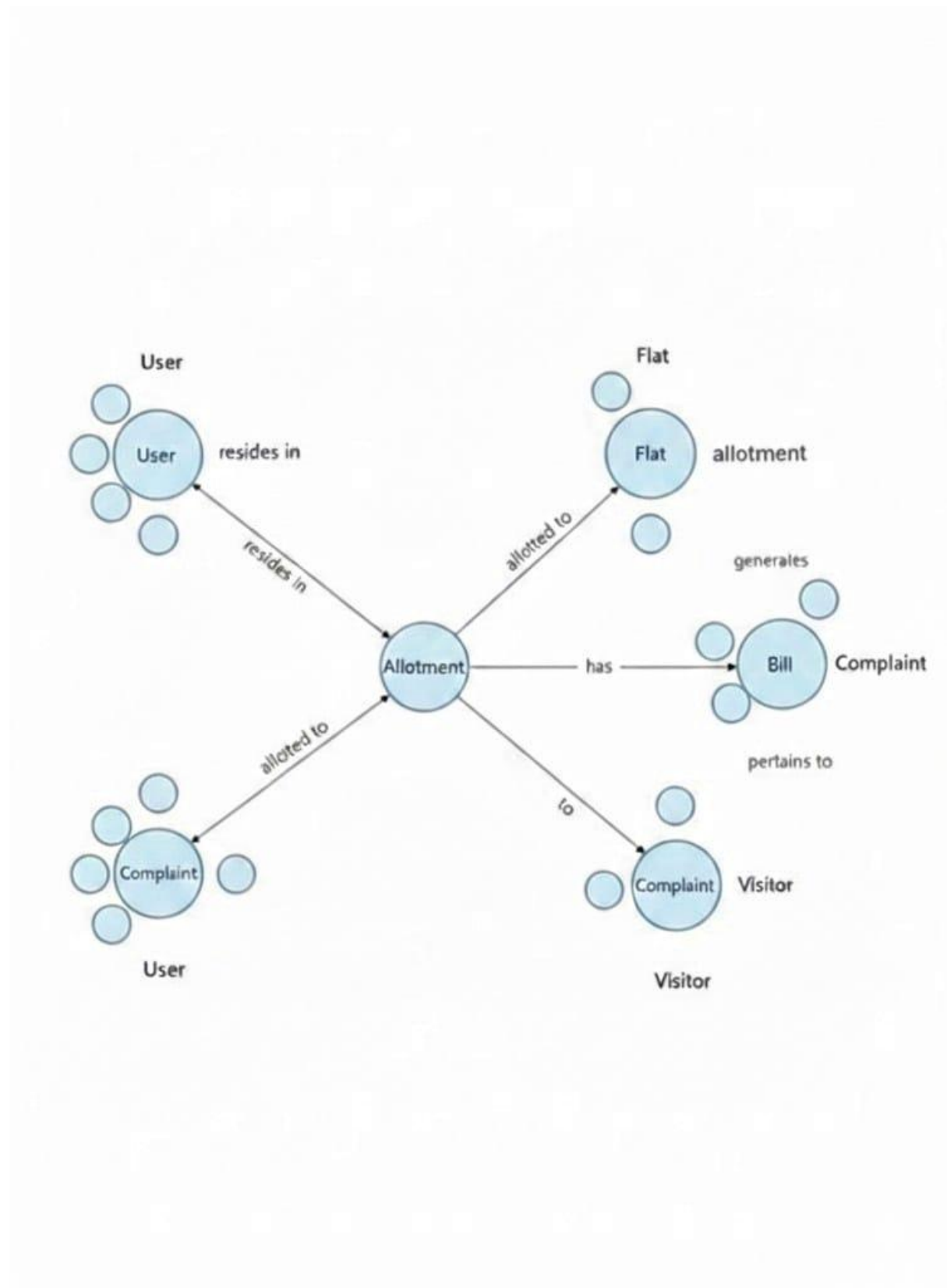
6. Visitor

Attributes: id, flat_id, name, phone, address, person_to_meet, reason, in_datetime, out_datetime

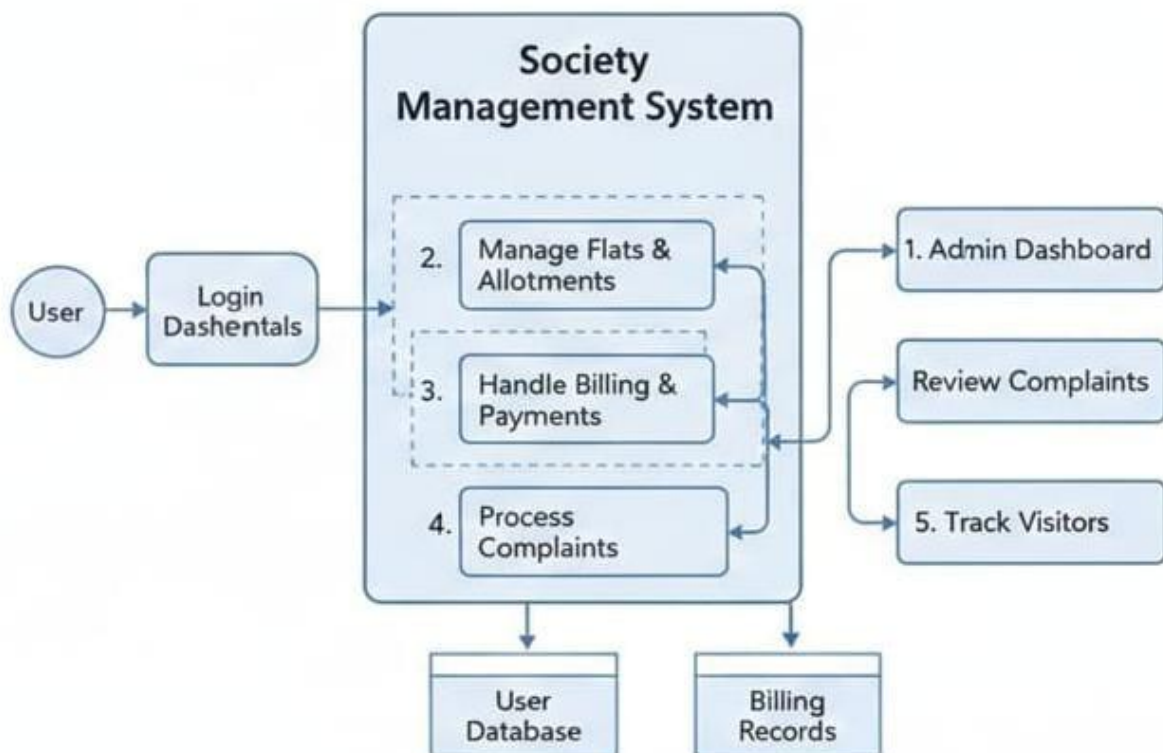
Purpose: Maintains a security log of external individuals entering the society premises and which units they are visiting.

2.5 ER DIAGRAM (UML)

The Entity-Relationship (ER) Diagram using UML notation illustrates how the system entities are related.



2.6 DATA FLOW



This structured approach ensures data consistency, normalization, and relational integrity, enabling smooth functioning of the system.

Phase	Task Description	W1	W2	W3	W4	W5	W6	W7	W8
Deployment	XAMPP Configuration & Final Review								█
Documentation	Final Project Report & Presentation								█

Detailed Phase Breakdown

Week 1-2 (Planning & Design):

Focuses on identifying user requirements (Admin vs. Resident) and mapping out the **MySQL database design**. This is where the relationships between flats, users, and bills are finalized.

Week 3-5 (Core Development):

The heavy lifting of the project. We establish the **PDO connection** and build the central `action.php` controller. Simultaneously, the **Bootstrap-based UI** is integrated to ensure the system is responsive.

Week 6 (Specialized Modules):

Development of the more complex features like the **Visitor Tracking system** (logging in/out) and the **Complaint Redressal workflow**.

Week 7 (Quality Assurance):

Rigorous testing of the security measures, ensuring **Bcrypt hashing** works correctly and that session-based access control prevents unauthorized entry.

Week 8 (Handover):

Finalizing the project report, generating the **ER Diagrams**, and preparing the system for demonstration.

2.7 PROJECT SCHEDULING: GANTT CHART

The Gantt chart below illustrates the 8-week development cycle for the **Society Management System**. This structured timeline ensures that the transition from a manual system to a digital PHP-MySQL environment is executed logically, allowing for a dedicated phase of testing before final delivery.

ID	Task Name	Durati on	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8
1. 0	Requirement Analysis	1 Week	■							
1. 1	Stakeholder Interviews (Admin/Residents)		■							
2. 0	System Design	1 Week		■						
2. 1	Database Schema (MySQL) & ERD			■						
2. 2	UI/UX Wireframing (Bootstrap)			■						
3. 0	Core Development	3 Weeks			■	■	■			
3. 1	Database Connection & User Auth				■					
3. 2	Flat & Resident Allotment Logic					■				
3. 3	Billing & Payment Module						■			

ID	Task Name	Durati on	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8
4. 0	Security & Support Modules	1 Week						■		
4. 1	Visitor Tracking & Complaints System							■		
5. 0	Quality Assurance	1 Week							■	
5. 1	Security Audit (Hashing/SQLi checks)								■	
6. 0	Deployment & Documentation	1 Week								■
6. 1	Final Report & Project Presentation									■

Visual Key

W1 - W2: Blueprints Phase. Focuses on the "Data Design" and "User Requirements" outlined in earlier sections.

W3 - W5: Execution Phase. The core PHP scripting and MySQL integration take place here.

W6 - W7: Security Hardening. Critical for society safety, focusing on visitor logs and database encryption.

W8: Deployment. Moving the system from a local development environment to a presentation-ready state.

2.8 HARDWARE REQUIREMENTS

The **Society Management System** is designed to be a lightweight yet robust web application. Because the system follows a client-server architecture, the hardware requirements are divided into two categories: the **Development/Server Side** (where the database and PHP engine reside) and the **Client Side** (where the administrator or resident accesses the portal).

2.8.1 Server-Side Requirements (Host Machine)

The server must be capable of running the **WAMP/XAMPP stack** smoothly to handle concurrent database queries and session management.

Processor: Intel Core i3 or equivalent (Minimum); Intel Core i5 or higher (Recommended for faster query processing).

RAM: 4 GB (Minimum); 8 GB or more (Recommended to handle multiple simultaneous resident logins).

Hard Disk Space: 500 MB for the application files and initial database; 10 GB+ recommended for long-term storage of visitor logs and billing history.

Architecture: 32-bit or 64-bit systems.

Internet/Network Connection: A stable Local Area Network (LAN) for society-wide access or a high-speed internet connection for cloud hosting.

2.8.2 Client-Side Requirements (User/Admin)

One of the primary advantages of this system is that it requires minimal hardware on the user's end, as most processing is done on the server.

Device: Any modern Desktop, Laptop, Tablet, or Smartphone.

Processor: Basic dual-core processor or modern mobile SoC.

RAM: 2 GB (Minimum).

Input Devices: Standard Keyboard and Mouse for the Administrator; Touchscreen or keyboard for Residents/Security Guards.

Display: Minimum resolution of **1024x768** for the Admin Dashboard; the interface is responsive and adapts to smaller mobile screens for residents.

2.9 Software Requirements

To ensure the system functions correctly, specific software environments must be established. The project relies on open-source technologies that are widely compatible.

Operating System: Windows 10/11, Linux (Ubuntu/CentOS), or macOS.

Web Server: Apache 2.4 (included in XAMPP/WAMP).

Server-Side Language: PHP 7.4 or 8.x (using PDO for secure database interaction).

Database: MySQL 5.7 or MariaDB 10.x.

Web Browser: Modern browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.

Development Tools: Visual Studio Code, Sublime Text, or Notepad++ for coding; phpMyAdmin for database management.

2.10 MODULE DIVISION

The **Society Management System** is architected into several interconnected modules. This modular approach ensures that the system is scalable, easy to maintain, and provides a clear separation of concerns between administrative duties and resident privileges.

The system is divided into the following key modules:

1. Authentication and Authorization Module

This is the gateway to the system. It handles user verification and session management.

Functionality: Provides secure login for both Admins and Residents, implements password hashing (Bcrypt), and maintains the `$_SESSION` state to ensure users only access pages authorized for their specific role.

2. User & Allotment Management Module

This module serves as the primary data management hub for people and property.

Functionality: Allows the Admin to register new residents and manage physical flat data (Block, Floor, Flat Number). The **Allotment** sub-module links these two, tracking move-in/move-out dates and maintaining a history of occupancy.

3. Financial & Billing Module

The core financial engine of the application.

Functionality: Admins can generate monthly maintenance bills for each flat. Residents can view their specific billing history, track outstanding balances, and update payment status once dues are cleared via cash or bank transfer.

4. Visitor Management Module

A security-focused module designed for real-time tracking of movement within the society.

Functionality: Security guards or admins can log guest entries (name, phone, reason for visit). The system records precise timestamps and tracks which flat the visitor is entering, replacing manual paper logbooks.

5. Grievance (Complaints) Module

This module facilitates communication regarding maintenance issues.

Functionality: Residents can file complaints (e.g., plumbing, electrical) through their portal. Admins receive these in a centralized dashboard where they can assign workers, add comments, and update the status from "Pending" to "Resolved."

6. Notification & Communication Module

Used for disseminating information across the community.

Functionality: This allows the management to broadcast important notices or individual alerts (e.g., bill reminders) to specific users, ensuring that critical information is seen upon login.

7. Reporting & Analytics Module

The data extraction layer of the system.

Functionality: Provides the Admin with filtered views and data exports. It can generate specific reports for monthly collections, visitor frequency, and complaint resolution rates, assisting in society audits and decision-making.

Chapter 3: Data Dictionary

3.1 DATA DICTIONARY

The success of the **Society Management System** depends heavily on how data is defined and managed. This section outlines the structural backbone of the database through a comprehensive Data Dictionary and an analysis of its importance in the development lifecycle.

A Data Dictionary is a centralized repository of metadata that describes the characteristics of the data stored in the database. It defines the names, types, lengths, and relationships of all fields within the tables.

Users (Authentications & Profiles)

Field Name	Data Type	Constraint	Description
id	INT	Primary Key	Unique ID for each system user.
name	VARCHAR(255)	Not Null	Full name of the resident or admin.
email	VARCHAR(255)	Unique	Login email address.
password	VARCHAR(255)	Not Null	Hashed password (Bcrypt).
role	ENUM	Not Null	Defines access level: 'Admin' or 'User'.

Flats (Property Details)

Field Name	Data Type	Constraint	Description
id	INT	Primary Key	Unique ID for each residential unit.
flat_no	VARCHAR(50)	Not Null	Specific number of the flat (e.g., 101).
floor	INT	Not Null	Floor level within the block.
block	VARCHAR(50)	Not Null	Block name or letter (e.g., A, B).

Bills (Financial Records)

Field Name	Data Type	Constraint	Description
id	INT	Primary Key	Unique ID for each transaction/invoice.
flat_id	INT	Foreign Key	Links the bill to a specific flat.
amount	DECIMAL(10,2)	Not Null	Total maintenance amount due.
status	ENUM	Default 'Unpaid'	Current state: 'Paid' or 'Unpaid'.

3.2 PURPOSE OF DATA DICTIONARY

The Data Dictionary serves as the "source of truth" for developers, analysts, and stakeholders. In the context of the **Society Management System**, its purpose extends far beyond mere documentation; it is a critical tool for ensuring system integrity and longevity.

1. Ensuring Data Integrity and Consistency

One of the primary purposes is to prevent data anomalies. By defining strict data types (e.g., ensuring amount is always a **DECIMAL** and not a **STRING**), the dictionary prevents the entry of invalid data that could crash the financial module. It ensures that every developer working on the project uses the same naming conventions, such as using `flat_id` consistently across the Bills, Allotments, and Complaints tables.

2. Facilitating Database Normalization

The dictionary helps in identifying redundant data. During the design phase, the dictionary makes it clear if the same information (like a resident's phone number) is being stored in multiple places. This allows the architect to normalize the database, reducing storage space and ensuring that an update to a user's profile reflects instantly across all modules, from visitor logs to billing receipts.

3. Enhancing Security and Access Control

By defining the role attribute in the user table within the dictionary, we establish the foundation for **Role-Based Access Control (RBAC)**. The dictionary specifies that only certain fields should be editable by residents, while others (like `paid_status`) remain exclusive to the Administrator. This clarity is essential when writing the PHP logic to handle session permissions.

4. Streamlining Maintenance and Scalability

As a society grows, the system may need new features like "Amenity Booking" or "Vehicle Tracking." A well-documented Data Dictionary allows future developers to understand the existing schema without having to reverse-engineer the code. It provides a roadmap for how new tables should link to existing ones (via Foreign Keys), ensuring the system can scale without breaking the core architecture.

5. Effective Communication Tool

Finally, it acts as a bridge between technical and non-technical stakeholders. If a society committee member asks how visitor data is stored, the Data Dictionary provides a clear, human-readable explanation of what information is captured (timestamps, reasons, contact info) without requiring the member to read SQL queries.

3.3 CORE ENTITIES IN THE DATA DICTIONARY

The Data Dictionary for the **Society Management System** is organized around six core entities. These entities represent the fundamental "objects" or "actors" within the system. By defining these core entities, we establish the boundaries of the system and how information flows between the administrative office and the residents.

1. The User Entity

The User entity is the primary actor. It contains the essential metadata required for authentication and personalization.

- **Key Attributes:** User_ID, Username, Hashed_Password, User_Role.
- **Significance:** It serves as the parent entity for almost all transactions. Whether a complaint is filed or a bill is paid, the system references the User_ID to maintain accountability.

2. The Flat Entity

The Flat entity represents the physical infrastructure of the society.

- **Key Attributes:** Flat_ID, Block_Name, Floor_Number, Flat_Type (e.g., 1BHK, 3BHK).
- **Significance:** This entity is static but foundational. It allows the administrator to visualize the occupancy of the building and serves as the anchor for the "Allotment" process.

3. The Allotment Entity

This is a logical entity that represents the relationship between a **User** and a **Flat**.

- **Key Attributes:** Allotment_ID, User_Reference, Flat_Reference, Move_In_Date.
- **Significance:** This entity is crucial for historical records. It ensures that the system knows exactly who was responsible for a specific flat during a specific billing cycle, even after a resident has moved out.

4. The Bill Entity

The Bill entity manages the financial state of the society.

- **Key Attributes:** Bill_ID, Total_Amount, Due_Date, Payment_Status.
- **Significance:** It tracks the receivables of the society. The data dictionary defines strict constraints on the Payment_Status (e.g., restricted to 'Pending', 'Paid', or 'Overdue') to ensure accurate financial reporting.

5. The Visitor Entity

The Visitor entity is the cornerstone of the security module.

- **Key Attributes:** Visitor_ID, Visitor_Name, In_Time, Out_Time, Host_Flat_ID.
- **Significance:** Unlike other entities that focus on residents, this entity captures external data. It ensures that every person entering the gate is digitally linked to a physical location within the society.

6. The Complaint Entity

The Complaint entity facilitates the maintenance workflow.

- **Key Attributes:** Complaint_ID, Description, Urgency_Level, Resolution_Status.
- **Significance:** It transforms a resident's verbal grievance into a trackable data point. The dictionary defines a Master_Comment field specifically for the Admin to close the feedback loop once a task is completed.

3.4 DATA RELATIONSHIPS

The Society Management System is built upon a relational database model where data consistency is maintained through primary keys, foreign keys, and defined constraints. The following section explains the logical connections between different entities.

1. Entity Relationship Overview

The system architecture revolves around the **Flat** entity as the central node. Most other entities—such as Visitors, Bills, and Allotments—are dependent on the existence of a flat record.

Users and Allotments (1:N and M:N Logic)

The relationship between a User and a Flat is managed through the **Allotments** table.

- **Logic:** While a resident is a "User," they only gain "Residency" status when an entry exists in the allotments table.
- **Relationship:** This is effectively a many-to-many relationship managed by a bridge table. One user can have multiple allotment histories (moving from one flat to another), and one flat can have a history of multiple users.

Flats and Visitors (One-to-Many)

This is a core security relationship defined in the visitors.php module.

- **Relationship: 1:N.** One flat can have many visitors logged against it.
- **Integrity:** Every visitor record must contain a flat_id. In database.sql, this is enforced with a foreign key: `CONSTRAINT visitors_ibfk_1 FOREIGN KEY (flat_id) REFERENCES flats(id).`

Users and Complaints (One-to-Many)

This relationship defines the service-request flow.

- **Relationship: 1:N.** One user can file multiple complaints.
- **Data Flow:** When a user logs in, the system uses their session_id to filter the complaints table, ensuring they only see their own tickets, while the Admin sees all complaints linked to various users.

2. Referential Integrity and Cascading Rules

To ensure the database remains clean and free of "orphaned" records (data pointing to a non-existent ID), the following cascading rules are implemented:

- **On Delete Cascade:** As seen in your database.sql file, if a **Flat** is deleted from the system, all associated **Visitors**, **Bills**, and **Allotments** are automatically deleted by the database engine (InnoDB).
- **Security Impact:** This ensures that if a user is removed (e.g., they leave the society), their private data and sensitive logs do not remain in the system indefinitely, supporting data privacy principles.

3. Data Flow in Relationships

The following table summarizes how the data flows through these relationships during a standard system operation:

Trigger Action	Primary Table	Related Table	Outcome
New Visitor Entry	visitors	flats	Validates flat_id exists before allowing entry.
Generating a Bill	bills	flats	Links the financial charge to a specific physical unit.
Resolving Complaint	complaints	users	Updates status and notifies the linked user_id.
User Deletion	users	allotments	Automatically clears the occupancy history for that user.

3.5 REFLECTIONS AS A STUDENT

Developing the Data Dictionary and the underlying system architecture was a transformative experience in my journey as a computer science student. It shifted my perspective from seeing software as merely visual screens to understanding it as a complex information ecosystem.

The following key lessons highlight my growth during this phase:

The Foundation Over the Façade:

Initially, I was eager to jump straight into designing the User Interface using Bootstrap, assuming that a good-looking dashboard was the mark of a successful project. However, I soon realized that without a rigorously defined data foundation, even the most beautiful UI is hollow. A button to "Generate Bill" or "Log Visitor" is useless if the underlying database schema cannot reliably store or retrieve that specific data point.

The Power of Normalization:

Designing the tables for the **Society Management System** taught me the practical necessity of database normalization. In my early drafts, I attempted to store resident details directly within the Visitor or Complaint tables for ease of access. I quickly learned that this redundancy leads to "Update Anomalies" where changing a phone number in one place wouldn't reflect in another. Separating these into distinct, linked entities like Users, Flats, and Allotments made the backend logic significantly cleaner and more efficient.

Precision in Naming:

I discovered that coding is as much about communication as it is about logic. Initially, I used generic names like ID or Date for multiple tables.

Database as the Backbone:

This stage of the project made me appreciate the silent work of the database. I realized that behind every seamless login and every generated report, there is a solid backbone of MySQL tables quietly ensuring data integrity and security.

In conclusion, the data dictionary not only served as a technical blueprint but also as a learning milestone, helping me understand the importance of structured database design in real-world applications.

Chapter 4: Implementation & Testing

The implementation phase transforms the theoretical designs into a functioning Society Management System (SMS). This chapter details the rigorous testing methodologies employed to ensure the system is stable, secure, and user-friendly, followed by a breakdown of the core code structure that powers the application.

TESTING

Testing serves as the quality control filter for the development cycle. The objective is to identify bugs, verify security protocols, and ensure that the software meets the requirements of a residential society environment.

4.1 UNIT TESTING

Unit testing is the first line of defense, focusing on the smallest possible parts of the code. Instead of looking at the whole dashboard, this phase examines individual functions and logic gates to ensure they behave exactly as expected.

Logic Verification:

Testing focused on the mathematical accuracy of the billing engine. This ensured that the system correctly calculated totals and handled decimal points without rounding errors that could lead to financial discrepancies.

Input Boundaries:

Every form field was subjected to boundary testing. This involved entering data that the system shouldn't allow—such as entering a text string into a phone number field or leaving a mandatory password field blank—to verify that the validation logic correctly caught and blocked these errors.

Security Integrity:

The unit tests verified that the password encryption process was functional. Testing ensured that the `password_hash` function created a unique string every time and that the system could reliably verify that hash during the login process without failing.

4.2 INTEGRATION TESTING

Once the individual units were confirmed to be working, the focus shifted to integration testing. This phase examines the alignment between different layers of the 3-tier architecture. It ensures that the frontend, the backend logic, and the database are all communicating without data loss.

The AJAX Bridge:

A major part of integration testing involved verifying that the jQuery scripts correctly packaged user data and sent it to the server-side controller. This "asynchronous" link is what allows the system to save data without refreshing the page, and testing ensured that the server always returned a clear success or error signal.

Database Persistence:

Testing confirmed that when the backend reported a success, the data was actually written to the MySQL disk. This was verified by cross-referencing the frontend view with the raw tables in phpMyAdmin after every major action like adding a flat or updating a resident's profile.

Role-Based Security:

Integration tests were used to confirm that the session management system was airtight. The system was tested to ensure that if an unauthorized user tried to bypass the UI and type a direct link to the admin settings, the backend would detect the missing credentials and immediately redirect the user to the login page.

4.3 BETA TESTING

Beta testing simulates the actual experience of a society administrator using the system for a full day of work. This phase is about usability and stability in a fluid environment rather than just technical correctness.

The User Experience (UX) Test:

This phase involved observing how non-developers navigated the system. It led to the observation that while the search bar worked perfectly, users needed more visual feedback. As a result, status messages were added to show when a bill was successfully saved or a visitor was logged out.

Stress and Edge Cases:

Beta testing involved "stressing" the system by performing actions out of order—such as trying to generate a bill for a flat that had no resident assigned. These tests allowed for the creation of more robust error-handling code that guides the user back to the correct workflow instead of letting the application crash.

Browser and Device Trials:

Since society staff might use various devices, the beta phase tested the dashboard on Google Chrome, Firefox, and mobile Safari. This ensured that the Bootstrap grid layout remained clean and that all buttons remained large enough to be clickable on smaller touchscreens.

4.4 CODE DESCRIPTION

The technical foundation of the Society Management System is built upon a balanced integration of three core technologies: PHP for server-side logic, SQL for data management, and CSS for visual structure. The codebase is organized to ensure that each language handles a specific responsibility, creating a system that is both secure and easy to navigate.

The Logic Layer: PHP (Hypertext Preprocessor)

PHP serves as the brain of the application, managing the communication between the user and the server. The architecture follows a centralized approach where most of the heavy lifting occurs in a primary controller file, often named `action.php`.

From a structural standpoint, PHP handles the following critical tasks:

Routing and Requests:

When a user clicks a button, PHP identifies the specific intent through `$_POST` variables. It uses switch-case or conditional logic to route these requests to the appropriate functions, such as registering a visitor or calculating a monthly maintenance bill.

Session Management:

Security is maintained through the `$_SESSION` superglobal. PHP scripts at the start of every page verify the user's login status and role. If an unauthorized attempt is made to access an administrative page, the PHP logic triggers an immediate redirect to the login screen.

Secure Database Interaction: The code utilizes PHP Data Objects (PDO) rather than older, less secure methods. This allows the system to use prepared statements, which act as a barrier between user input and the SQL database, preventing malicious code from being executed.

The Data Layer: SQL (Structured Query Language)

The storage and organization of society information are handled entirely by SQL. The database architecture is designed with a relational schema, meaning that different tables are linked together to prevent data duplication.

The SQL implementation includes:

Data Persistence:

Tables such as users, flats, allotments, visitors, and bills store every piece of information permanently. Each table uses a Primary Key (like `user_id`) to ensure every record is unique.

Relational Joins:

The power of SQL is utilized through JOIN queries. For instance, to display a resident's name alongside their flat number, the code executes a query that links the users table to the flats table through the allotments table.

Integrity Constraints:

SQL is configured with foreign key constraints. This ensures that if a flat is deleted, the system can automatically handle related records, preventing the database from becoming cluttered with "orphaned" data that no longer belongs to an active unit.

The Presentation Layer: CSS (Cascading Style Sheets)

The visual environment is governed by CSS, specifically through the Bootstrap 5 framework and the SB Admin theme. This layer is responsible for turning raw data into a professional-grade dashboard.

The CSS structure provides:

Responsive Grid System:

By using Bootstrap's pre-defined classes, the system ensures that the interface is "liquid." It automatically rearranges elements based on the device being used, ensuring that tables and forms remain usable on both mobile phones and desktop monitors.

Visual Communication:

Custom CSS is used to create color-coded status indicators. For example, CSS classes like badge-success and badge-danger are applied to billing statuses, allowing an administrator to instantly distinguish between paid and unpaid dues without reading every line of text.

UI Consistency:

The SB Admin template provides a global stylesheet that ensures all buttons, sidebars, and navigation bars have a uniform appearance, creating a cohesive user experience throughout the application.

The Interaction Layer: JavaScript and JSON

Connecting these three layers is a layer of JavaScript (jQuery) and JSON. This acts as the "glue" that makes the code feel like a modern application.

Asynchronous Communication (AJAX):

JavaScript captures the user's actions and sends them to the PHP backend in the background. This prevents the browser from having to reload the entire page every time a record is saved.

JSON Data Exchange:

Once the PHP and SQL have finished their work, they send a response back in JSON format. This lightweight data package tells the JavaScript whether the operation was a success or a failure, allowing the UI to update dynamically.

By strictly separating these languages into their respective roles, the system achieves a modular design. If the look of the site needs to change, only the CSS is modified; if the logic needs to

change, only the PHP is updated; and if the data structure needs to evolve, the SQL schema is adjusted all without breaking the other components of the system.

4.5 CODE (SAMPLE IMPLEMENTATION)

action.php (Condensed Logic)

This file handles all AJAX requests. I have used a more efficient way to handle the DataTables logic to keep it brief.

PHP

```
<?php
```

```
require_once 'config.php';
```

```
if(isset($_POST['action'])) {
```

```
    $action = $_POST['action'];
```

```
    // Generic function to handle DataTables Fetch
```

```
    function fetch_data($pdo, $query, $columns) {
```

```
        if(!empty($_POST["search"]["value"])) {
```

```
            $query .= ' WHERE ';
```

```
            $search_parts = [];
```

```
            foreach($columns as $col) $search_parts[] = "$col LIKE  
            '%" . $_POST["search"]["value"] . "%'";
```

```
            $query .= implode(" OR ", $search_parts);
```

```
        }
```

```
        if($_POST["length"] != -1) $query .= ' LIMIT ' . $_POST['start'] . ', ' . $_POST['length'];
```

```
        $stmt = $pdo->prepare($query);
```

```
        $stmt->execute();
```

```
        return $stmt->fetchAll();
```

```
    }
```

```
    if($action == 'fetch_users') {
```

```
        $cols = ['id', 'name', 'email', 'role'];
```

```

$data = fetch_data($pdo, "SELECT * FROM users", $cols);
$output = [];
foreach($data as $row) {
    $output[] = [
        "id" => $row["id"], "name" => $row["name"],
        "email" => $row["email"], "role" => $row["role"],
        "created_at" => $row["created_at"],
        "action" => '<a href="edit_user.php?id='.$row["id"].'" class="btn btn-sm btn-primary">Edit</a>'
    ];
}
echo json_encode(["data" => $output]);
}

if($action == 'fetch_visitors') {
    $query = "SELECT visitors.*, flats.flat_number FROM visitors INNER JOIN flats ON flats.id = visitors.flat_id";
    $data = fetch_data($pdo, $query, ['name', 'phone', 'person_to_meet']);
    $output = [];
    foreach($data as $row) {
        $status = ($row['is_in_out'] == 'in') ? '<span class="badge bg-danger">In</span>' : '<span class="badge bg-success">Out</span>';
        $output[] = [
            "id" => $row['id'], "flat" => $row['flat_number'], "name" => $row['name'],
            "phone" => $row['phone'], "status" => $status,
            "action" => '<a href="view_visitor.php?id='.$row["id"].'" class="btn btn-warning btn-sm">View</a>'
        ];
    }
    echo json_encode(["data" => $output]);
}
}

```

?>

database.sql (Schema Only)

This version contains only the table definitions (DDL), which are the most important for a report.

SQL

```
CREATE TABLE `users` (  
  `id` int(11) UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  `name` varchar(255) NOT NULL,  
  `email` varchar(255) UNIQUE NOT NULL,  
  `password` varchar(255) NOT NULL,  
  `role` enum('admin','user') NOT NULL,  
  `created_at` timestamp DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE `flats` (  
  `id` int(11) UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  `flat_number` varchar(50) NOT NULL,  
  `floor` varchar(50) NOT NULL,  
  `block_number` varchar(50) NOT NULL,  
  `flat_type` varchar(50) NOT NULL  
) ENGINE=InnoDB;
```

```
CREATE TABLE `visitors` (  
  `id` int(11) UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  `flat_id` int(11) UNSIGNED NOT NULL,  
  `name` varchar(255) NOT NULL,  
  `phone` varchar(30) NOT NULL,  
  `in_datetime` datetime NOT NULL,  
  `out_datetime` datetime DEFAULT NULL,  
  `is_in_out` enum('in','out') NOT NULL,  
  FOREIGN KEY (`flat_id`) REFERENCES `flats`(`id`) ON DELETE CASCADE
```

```
) ENGINE=InnoDB;
```

```
CREATE TABLE `complaints` (  
  `id` int(11) UNSIGNED PRIMARY KEY AUTO_INCREMENT,  
  `user_id` int(11) UNSIGNED NOT NULL,  
  `flat_id` int(11) UNSIGNED NOT NULL,  
  `complaint_text` text NOT NULL,  
  `status` enum('Pending','Resolved') DEFAULT 'Pending',  
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

3. visitors.php (Cleaned Module)

The visitor tracking interface focuses on the DataTables integration.

PHP

```
<?php  
require_once 'config.php';  
if (!isset($_SESSION['user_id'])) { header('Location: logout.php'); exit(); }  
  
if(isset($_GET['action'], $_GET['id']) && $_GET['action'] == 'delete') {  
    $pdo->prepare("DELETE FROM visitors WHERE id = ?")->execute([$_GET['id']]);  
    $_SESSION['success'] = 'Visitor Data removed';  
    header('location:visitors.php');  
}  
include('header.php');  
?  
  
<div class="container-fluid px-4">  
    <h1 class="mt-4">Visitor Management</h1>  
    <div class="card mb-4">  
        <div class="card-header"><i class="fas fa-table me-1"></i> Visitor Logs</div>  
        <div class="card-body">  
            <table id="visitors-table" class="table table-bordered table-striped">  
                <thead>
```



```

<tr><th>ID</th><th>Flat</th><th>Name</th><th>Phone</th><th>Status</th><th>Action</th></tr>

    </thead>

</table>

</div>

</div>

</div>

<script>
$(document).ready(function() {
    $('#visitors-table').DataTable({
        "processing": true, "serverSide": true,
        "ajax": { url: 'action.php', method: "POST", data: {action: 'fetch_visitors'} },
        "columns": [
            { "data": "id" }, { "data": "flat" }, { "data": "name" },
            { "data": "phone" }, { "data": "status" }, { "data": "action" }
        ]
    });
});
</script>
<?php include('footer.php'); ?>

```

styles.css (Core Theme Styles)

Since the original SB Admin CSS is thousands of lines, this version includes only the essential custom styles for the dashboard and cards.

CSS

```

:root {
    --bs-primary: #0d6efd;
    --bs-success: #198754;

```

```

--bs-danger: #dc3545;
}

body { font-family: 'Inter', sans-serif; background-color: #f8f9fa; }

/* Dashboard Cards */

.card { border: none; border-radius: 10px; box-shadow: 0 0.125rem 0.25rem rgba(0, 0, 0, 0.075); margin-bottom: 1.5rem; }

.card-header { background-color: #fff; border-bottom: 1px solid #e3e6f0; font-weight: bold; }

/* Sidebar Styles */

#layoutSidenav_nav { width: 225px; transition: transform 0.15s ease-in-out; }

.sb-sidenav { display: flex; flex-direction: column; height: 100%; background-color: #212529; color: rgba(255, 255, 255, 0.5); }

/* Badges and Tables */

.badge { padding: 0.5em 0.8em; font-size: 0.85rem; }

.dataTable-table > thead > tr > th { vertical-align: bottom; border-bottom: 2px solid #dee2e6; }

/* Responsive Adjustments */

@media (max-width: 768px) {
  #layoutSidenav_nav { transform: translateX(-225px); }
}

```

index.php (Secure Login)

Ensures password verification and session initialization.

PHP

```
<?php
```

```
require_once 'config.php';
```

```
if (isset($_SESSION['user_id'])) { header('Location: dashboard.php'); exit(); }
```

```

if(isset($_POST['btn_login'])) {
    $stmt = $pdo->prepare("SELECT * FROM users WHERE email = ?");
    $stmt->execute([$_POST['email']]);
    $user = $stmt->fetch();

    if ($user && password_verify($_POST['password'], $user['password'])) {
        $_SESSION['user_id'] = $user['id'];
        $_SESSION['user_role'] = $user['role'];
        header('Location: dashboard.php');
    } else {
        $error = "Invalid Credentials";
    }
}

?>

<!DOCTYPE html><html><head><title>Login</title><link rel="stylesheet"
href="styles.css"></head>

<body class="bg-primary">

    <form method="POST" class="container mt-5 p-5 bg-white rounded shadow" style="max-
width: 400px;">

        <h2>Society Management Login</h2>

        <?php if(isset($error)) echo "<div class='alert alert-danger'>$error</div>"; ?>

        <input type="email" name="email" class="form-control mb-3" placeholder="Email"
required>

        <input type="password" name="password" class="form-control mb-3"
placeholder="Password" required>

        <button name="btn_login" class="btn btn-primary w-100">Login</button>

    </form>

</body></html>

```

dashboard.php (Metric Summary)

Uses simple SQL counts to populate the admin stats.

PHP

```
<?php
```

```
require_once 'config.php';

if (!isset($_SESSION['user_id']) || $_SESSION['user_role'] !== 'admin') { header('Location:
logout.php'); exit(); }
```

```
$stats = [

    'Flats' => $pdo->query("SELECT COUNT(*) FROM flats")->fetchColumn(),

    'Users' => $pdo->query("SELECT COUNT(*) FROM users")->fetchColumn(),

    'Visitors' => $pdo->query("SELECT COUNT(*) FROM visitors")->fetchColumn(),

    'Complaints' => $pdo->query("SELECT COUNT(*) FROM complaints WHERE
status='Pending'")->fetchColumn()

];
```

```
include('header.php');
```

```
?>
```

```
<div class="container-fluid px-4">
```

```
    <h1 class="mt-4">Dashboard</h1>
```

```
    <div class="row mt-4">
```

```
        <?php foreach($stats as $label => $val): ?>
```

```
            <div class="col-xl-3 col-md-6">
```

```
                <div class="card bg-white text-dark mb-4 p-3">
```

```
                    <h5>Total <?php echo $label; ?></h5>
```

```
                    <h2 class="fw-bold"><?php echo $val; ?></h2>
```

```
                </div>
```

```
            </div>
```

```
        <?php endforeach; ?>
```

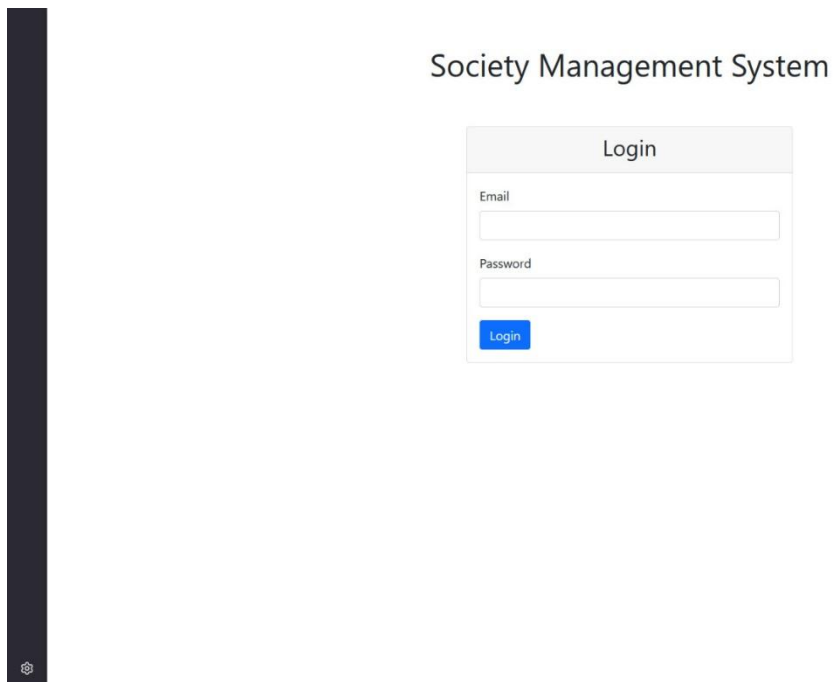
```
    </div>
```

```
</div>
```

```
<?php include('footer.php'); ?>
```

4.6 SCREENSHOTS

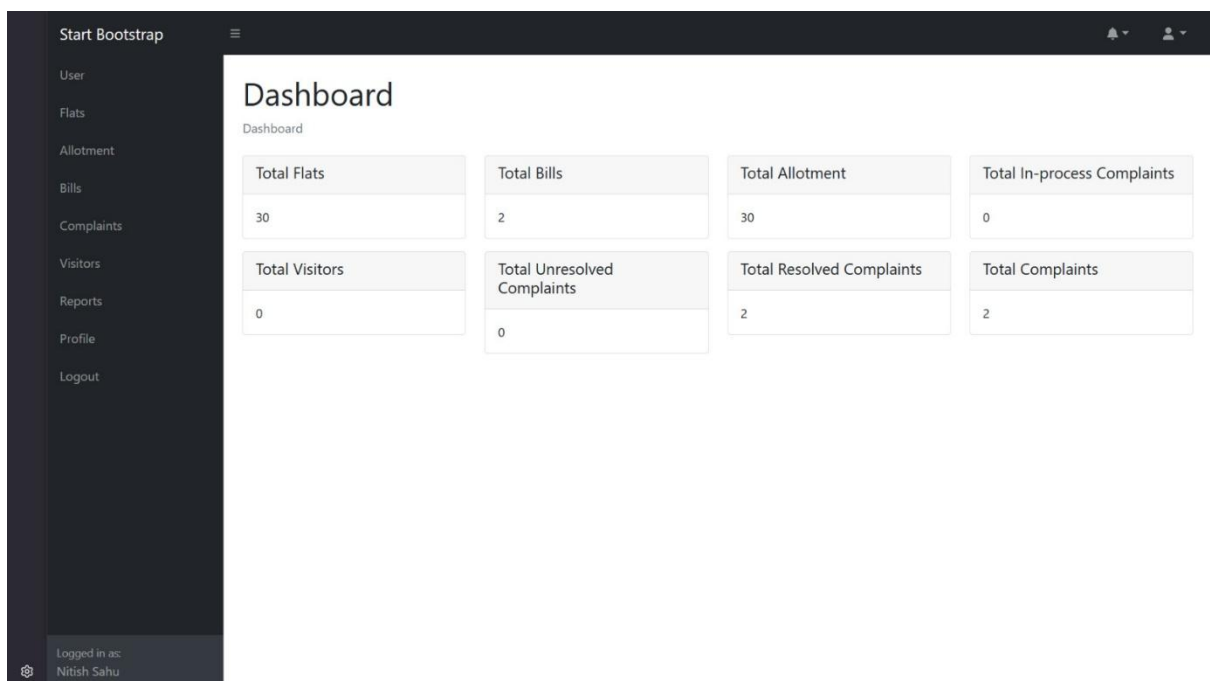
Login Page : Secure User Authentication Portal



The screenshot shows the login page of the Society Management System. It features a dark vertical sidebar on the left with a gear icon at the bottom. The main content area is white and contains the title "Society Management System" at the top. Below the title is a "Login" form with two input fields: "Email" and "Password". A blue "Login" button is positioned below the password field.

Login	
Email	
Password	
<button>Login</button>	

Dashboard : Centralized Administrative Dashboard



The screenshot shows the dashboard of the Society Management System. It features a dark vertical sidebar on the left with a menu of options: "Start Bootstrap", "User", "Flats", "Allotment", "Bills", "Complaints", "Visitors", "Reports", "Profile", and "Logout". The main content area is white and contains the title "Dashboard" and a subtitle "Dashboard". Below the title are eight summary cards arranged in a 2x4 grid, each showing a total value.

Total Flats	Total Bills	Total Allotment	Total In-process Complaints
30	2	30	0

Total Visitors	Total Unresolved Complaints	Total Resolved Complaints	Total Complaints
0	0	2	2

Logged in as: Nitish Sahu

User Management : Flat Inventory & Asset Management

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

User Management

[Dashboard](#) / [User Management](#)

Users Management

Add

Show 10 entries

Search:

ID	Name	Email	Role	Created At	Action
1	Nitish Sahu	admin@sms.com	admin	2026-02-24 14:45:53	Edit Delete
3	Rahul Verma	rahulverma@sms.com	user	2026-03-06 17:36:34	Edit Delete
4	Priya Patel	priyapatel@sms.com	user	2026-03-06 17:37:36	Edit Delete
5	Neha Gupta	nehagupta@sms.com	user	2026-03-06 17:38:04	Edit Delete
6	Sneha Singh	snehasingh@sms.com	user	2026-03-06 17:38:40	Edit Delete
7	Vikram Rao	vikramrao@sms.com	user	2026-03-06 17:39:09	Edit Delete
8	Rohit Desai	rohitdesai@sms.com	user	2026-03-06 17:40:06	Edit Delete
9	Sanjay Kumar	sanjaykumar@sms.com	user	2026-03-06 17:40:51	Edit Delete
10	Rajesh Reddy	rajeshreddy@sms.com	user	2026-03-06 17:41:21	Edit Delete
11	Anil Joshi	aniljoshi@sms.com	user	2026-03-06 17:42:14	Edit Delete

Showing 1 to 10 of 30 entries

[Previous](#) [1](#) [2](#) [3](#) [Next](#)

Flats Management

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Flats Management

[Dashboard](#) / [Flats Management](#)

Flats Management

Add

Show 10 entries

Search:

ID	Flats Number	Floor Number	Block Number	Type	Created At	Action
1	101	1	A	1 BHK Flat	2026-02-20 17:07:45	Edit Delete
2	102	1	A	1 BHK Flat	2026-03-06 18:18:39	Edit Delete
3	103	1	A	1 BHK Flat	2026-03-06 18:18:55	Edit Delete
4	104	1	A	1 BHK Flat	2026-03-06 18:19:08	Edit Delete
5	201	2	A	1 BHK Flat	2026-03-06 18:19:24	Edit Delete
6	202	2	A	1 BHK Flat	2026-03-06 18:19:33	Edit Delete
7	203	2	A	1 BHK Flat	2026-03-06 18:19:47	Edit Delete
8	204	2	A	1 BHK Flat	2026-03-06 18:20:00	Edit Delete
9	301	3	A	1 BHK Flat	2026-03-06 18:20:17	Edit Delete
10	302	3	A	1 BHK Flat	2026-03-06 18:20:28	Edit Delete

Showing 1 to 10 of 30 entries

[Previous](#) [1](#) [2](#) [3](#) [Next](#)

Add Flats : For Adding More flats in Admin panel

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as: Nitish Sahu

Add Flats

[Dashboard](#) / [Flats Management](#) / Add Flats Management

Add Flats

Flat Number

Floor

Block Number

Type

Select Type

Add Flat

Allotments Management : Resident and Staff Directory

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as: Nitish Sahu

Allotments Management

[Dashboard](#) / Allotments Management

Allotments Management

Add

Show 10 entries

Search:

ID	Alloted to	Flat Number	Block Number	Type	Move in Date	Move Out Date	Created At	Action
1	Rahul Verma	101	A	1 BHK Flat	2026-01-01	0000-00-00	2026-02-22 18:23:46	Edit Delete
2	Siddharth Thakur	102	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:26:20	Edit Delete
3	Ravi Kulkarni	103	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:26:48	Edit Delete
4	Harish Tiwari	104	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:27:29	Edit Delete
5	Alok Chawla	201	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:27:51	Edit Delete
6	Dinesh Ahuja	202	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:28:40	Edit Delete
7	Naveen Jain	203	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:29:03	Edit Delete
8	Mohan Soni	204	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:29:31	Edit Delete
9	Vijay Patil	301	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:30:00	Edit Delete
10	Arjun Rathi	302	A	1 BHK Flat	2026-01-01	0000-00-00	2026-03-06 18:30:33	Edit Delete

Showing 1 to 10 of 30 entries

[Previous](#) [1](#) [2](#) [3](#) [Next](#)

Add Allotments

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Add Flats

[Dashboard](#) / [Allotments Management](#) / Add Flats Management

Add Allotments

User

-- Select User --

Flat

-- Select Flat --

Move In Date

dd / mm / yyyy

Move Out Date

dd / mm / yyyy

Add Allotment

Bills Management

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Bills Management

[Dashboard](#) / Bills Management

Bills Management

Add

Show 10 entries

Search:

ID	Bill Title	Flat Number	Bill Amount	Month	Paid Amount	Updated At	Action
3	Electricity Bill for January 2026	A - 101	350.00	2026-01	350.00	2026-02-22 18:26:25	View Edit Delete
5	Electricity Bill for February 2026	A - 101	425.00	2026-02	425.00	2026-03-02 18:22:06	View Edit Delete

Showing 1 to 2 of 2 entries

[Previous](#) [1](#) [Next](#)

Add Bill Data

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Add Bill Data

[Dashboard](#) / [Bills Management](#) / Add Bill Data

Add Bill Data

Bill Title

Flat Number

Amount

Month

Add Bill

Bill Report

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Bill Report

[Dashboard](#) / [Reports](#)

Report

Report For

Start Date

End Date

Generate Report

Complaints Management : Community Complaints & Resolution Portal

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Complaints Management

[Dashboard](#) / [Complaints Management](#)

Complaints Management

Show 10 entries

Search:

ID	User Name	Flat Number	Complaint	Status	Updated At	Action
1	Rahul Verma	A - 101	Testing Complaints with edit process	Resolve	2026-02-25 18:12:20	View
2	Rahul Verma	A - 101	At my floor common bulb is not working. Can you install new bulb there.	Resolve	2026-03-03 10:47:50	View

Showing 1 to 2 of 2 entries

Previous1Next

Start Bootstrap

Bills

Complaints

Visitors

Logout

Logged in as:
Max Bro

Complaints Management

[Dashboard](#) / [Complaints Management](#)

Complaints Management

Add

Show 10 entries

Search:

ID	User Name	Flat Number	Complaint	Status	Updated At	Action
1	Max Bro	A - 101	Testing Complaints with edit process	Resolve	2026-02-25 18:12:20	View
2	Max Bro	A - 101	At my floor common bulb is not working. Can you install new bulb there.	Resolve	2026-03-03 10:47:50	View

Showing 1 to 2 of 2 entries

Previous1Next

View Complaints

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

View Complaints

[Dashboard](#) / [Complaints Management](#) / View Complaints

View Complaint

User Name	Rahul Verma
Flat Number	A - 101
Complaints Details	Testing Complaints with edit process
Status	Resolved
Master Comment	Comment Date & Time - 2026-02-27 11:04:20 Comment - Testing Master in process comment Comment Date & Time - 2026-02-27 11:06:06 Comment - Testing Master Resolved Comment

Add Complaints

Start Bootstrap

Bills

Complaints

Visitors

Logout

Logged in as:
Max Bro

Add Complaints

[Dashboard](#) / [Complaints Management](#) / Add Complaints

Add Complaint

Complaint Description

Enter Complaint Description

Add Complaint

Visitor Management : Visitor Monitoring & Security Log

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Visitor Management

[Dashboard](#) / [Visitor Management](#)

Visitor Management

Add

Show 10 entries

Search:

ID	Flat Number	Visitor Name	Visitor Phone	Person to Meet	In Time	Out Time	Status	Action
No data available in table								

Showing 0 to 0 of 0 entries

PreviousNext

Add Visitors : Basic Details of Visitors and Stay Duration

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Add Visitors

[Dashboard](#) / [Visitors Management](#) / [Add Visitor](#)

Add Visitor

Flat Number

Select Flat

Visitor Name

Phone Number

Address

Person to Meet

Reason for Visit

In Date/Time

dd / mm / yyyy, --:-- --

Add Visitor

Bill Reports

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Bill Report

[Dashboard](#) / [Reports](#)

Report

Report For

Select

Start Date

dd / mm / yyyy

End Date

dd / mm / yyyy

Generate Report

User and Admin Profile

Start Bootstrap

User

Flats

Allotment

Bills

Complaints

Visitors

Reports

Profile

Logout

Logged in as:
Nitish Sahu

Profile

Edit Profile

Edit Profile

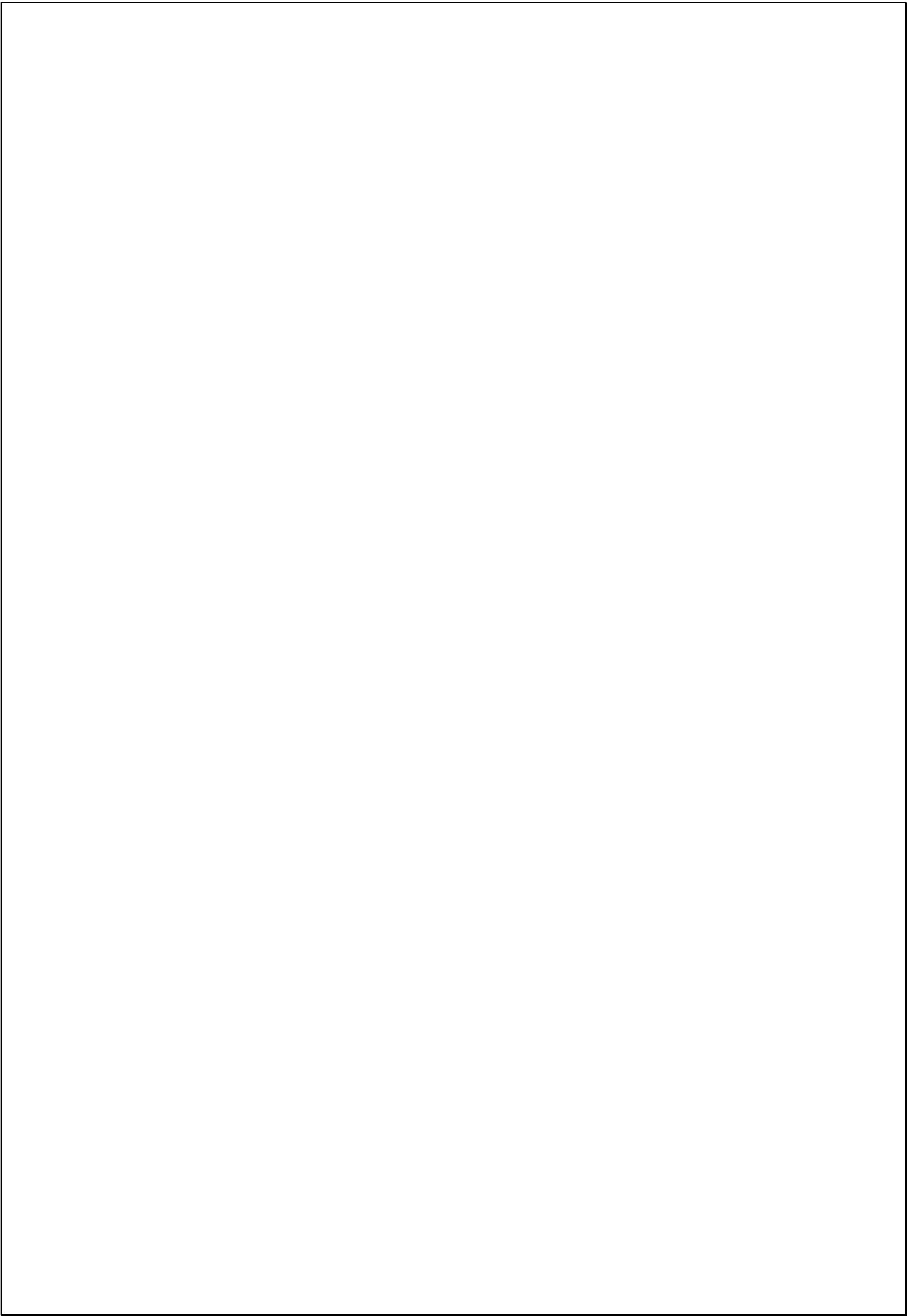
Name

Nitish Sahu

Email

admin@sms.com

Save



Chapter 5: System Architecture

The architecture of the Society Management System (SMS) follows a **Client-Server model**, specifically utilizing the **3-Tier Architecture** pattern. This chapter delves into the Presentation Layer (Frontend), exploring how the user interface interacts with the underlying logic and the structural components that ensure a seamless user experience.

5.1 FRONTEND ARCHITECTURE

The frontend of the Society Management System is the "face" of the application. It is designed to be **lightweight, responsive, and state-driven**. The architecture focuses on the **Presentation Layer**, which is responsible for transforming raw data from the MySQL database into a human-readable and interactive format.

The frontend architecture is built using a combination of **HTML5, CSS3 (via Bootstrap 5), and JavaScript (via jQuery)**. This stack ensures that the application is not only aesthetically pleasing but also functional across a variety of devices, from high-resolution desktop monitors in the society office to mobile smartphones used by residents on the go.

5.1.1 Components of the Frontend

The frontend is not a single monolithic file but a collection of modular components that work in unison. This modularity allows for easier maintenance and a consistent look and feel across the platform.

1. The Templating Engine (Header, Sidebar, Footer)

To maintain consistency, the frontend uses a modular templating system. Instead of rewriting the navigation menu for every page, the system utilizes a centralized `header.php` and `sidebar.php`.

Dynamic Sidebar:

The sidebar is a "smart" component. It checks the user's session role and renders different navigation links accordingly. For an Admin, it displays "Manage Users" and "Financial Reports"; for a Resident, it displays "My Bills" and "File Complaint."

The Content Area:

This is the dynamic portion of the screen that changes based on the user's selection, contained within a responsive Bootstrap container.

2. Bootstrap 5 Responsive Grid System

The core structural component of the frontend is the Bootstrap grid.

Responsiveness:

By using classes like `col-md-6` or `col-lg-4`, the frontend automatically rearranges elements based on the screen width. On a mobile device, a two-column form will stack vertically to maintain readability.

UI Components:

The system leverages Bootstrap's pre-built components such as **Cards** (to display summary statistics), **Modals** (for adding quick visitor logs without leaving the page), and **Alerts** (to show success or error messages after a form submission).

3. DataTables Integration (Dynamic Lists)

A significant component of the frontend architecture is the integration of the **jQuery DataTables** plugin.

Client-Side Interaction:

While the data resides in the database, the frontend component provides the user with an interactive interface to search, sort, and paginate through hundreds of visitor or billing records instantaneously.

AJAX Communication:

The DataTable component communicates with the backend via asynchronous requests, meaning the entire page does not need to refresh every time a user searches for a specific flat number.

4. Form Validation and User Input

The frontend acts as the first line of defense for data integrity.

HTML5 Validation:

Every input field uses attributes like `required`, `pattern`, and `type="email"` to ensure that data is formatted correctly before it ever reaches the server.

Visual Feedback:

Using CSS pseudo-classes, the frontend provides immediate visual cues—turning a text box border red if an input is invalid or green once a resident types a valid password.

5.1.2 Frontend Flow

The Frontend Flow describes the journey of a user request from the moment they click a button to the moment they receive visual feedback. This flow is governed by the principles of **Event-Driven Programming**.

Step 1: The Request Initiation (User Interaction)

The flow begins when a user interacts with a UI component. For example, an Admin clicks "Add Bill." The frontend architecture listens for this event. In the case of this system, many actions are handled by **jQuery event listeners**.

Step 2: Client-Side Processing and Data Packaging

Before sending a request to the server, the frontend packages the data. If it is a form submission, the `serialize()` function in jQuery gathers all the input values (Amount, Flat ID, Month) and converts them into a format that the backend `action.php` can understand.

Step 3: The Asynchronous Bridge (AJAX)

One of the most critical aspects of the frontend flow in this architecture is the use of **AJAX (Asynchronous JavaScript and XML)**.

- Instead of a traditional "Submit and Wait" flow, the frontend sends the data in the background.
- This allows the user to continue interacting with other parts of the dashboard while the "bill" is being processed by the server.

Step 4: Response Handling and DOM Manipulation

Once the server responds (usually with a JSON object), the frontend flow enters the "Success" or "Error" phase.

- **Success:** If the backend confirms the bill was created, the frontend uses JavaScript to dynamically update the **DOM (Document Object Model)**. It might clear the form, hide the modal, and append a new row to the data table without a page reload.
- **Error:** If the server rejects the request (e.g., duplicate bill), the frontend triggers a "Toast" notification or a Bootstrap alert to inform the user exactly what went wrong.

Step 5: State Persistence and Navigation

Finally, the frontend flow manages the user's "State." If a user logs in, the frontend ensures that the session information is reflected in the UI—showing the user's name in the top-right corner and ensuring that navigating back to the login page is prevented.

Summary of the Frontend Stack

Component	Technology	Purpose
Structure	HTML5	Defining the semantic layout of the management pages.
Styling	CSS3 / Bootstrap 5	Ensuring a professional look and mobile responsiveness.
Interactivity	JavaScript / jQuery	Handling button clicks, form submissions, and UI updates.
Dynamic Tables	DataTables.net	Providing advanced search/filter features for records.

Component	Technology	Purpose
Icons	FontAwesome 6	Providing visual cues for navigation and action buttons.

5.2 BACKEND ARCHITECTURE

The backend represents the logical layer of the system, handling processing, integration, and control flow.

The backend architecture of the **Society Management System (SMS)** serves as the "brain" of the application. While the frontend focuses on presentation, the backend is responsible for data processing, security enforcement, and business logic execution. This system utilizes a **Server-Side Scripting** approach powered by **PHP**, following a modular structure that ensures every request—whether it is generating a maintenance bill or logging a visitor—is handled with precision and security.

The architecture is built on the **WAMP/XAMPP stack**, leveraging the **Apache** web server to listen for client requests and the **PHP engine** to interpret scripts. The backend is designed to be stateless and efficient, relying on **PDO (PHP Data Objects)** for secure database communication, which provides a layer of abstraction that protects the society's sensitive data from common vulnerabilities.

5.2.1 Components of the Backend

The backend is composed of several critical layers that work in harmony to manage the society's operations:

1. The Controller Logic (action.php)

The heart of the backend is the centralized controller. In this architecture, `action.php` acts as a traffic controller. Instead of having dozens of individual files for every minor task, the system routes AJAX requests to this central file.

Request Parsing:

It identifies the "action" type (e.g., `fetch_visitors`, `add_flat`, `update_complaint`).

Execution: Based on the action, it executes specific PHP functions that interact with the database.

Response Generation:

It packages the results into a **JSON (JavaScript Object Notation)** format, which is then sent back to the frontend.

2. Data Access Layer (PDO & MySQL)

The backend does not interact with the database using raw, insecure queries. Instead, it utilizes the **PDO extension**.

Prepared Statements:

This component ensures that all user inputs are treated as data, not executable code, effectively neutralizing **SQL Injection** attacks.

Database Driver:

It provides the connection string that links the PHP scripts to the MySQL server, ensuring that the sms_db schema is accessible.

3. Session & Security Manager

This component handles the "state" of the user. Since HTTP is a stateless protocol, the backend uses **PHP Sessions** to remember who is logged in.

Role Verification:

Before executing an administrative task (like deleting a user), this component checks the \$_SESSION['role'] to ensure the requester has 'Admin' privileges.

Authentication Logic:

It manages the **Bcrypt hashing** process. When a user logs in, this component retrieves the hashed password from the database and uses password_verify() to authorize access.

4. Configuration & Environment (config.php)

This is the foundational component that defines the system's global parameters. It contains the database credentials, base URL settings, and error-reporting levels. It acts as the "Settings" menu for the backend, allowing the system to be migrated from a local XAMPP server to a live cloud server by changing just a few lines of code.

5.2.2 Backend Flow

The backend flow represents the internal logical sequence that occurs from the moment a request hits the server until a response is dispatched. This flow is designed to be synchronous in execution but asynchronous in its relationship with the user interface.

Step 1: Request Reception and Sanitization

The flow begins when the Apache server receives an HTTP POST or GET request. The backend immediately captures the global variables (\$_POST or \$_GET). The first priority is **sanitization**. The system checks for the presence of required tokens and ensures that the request is not coming from a cross-site forgery attempt.

Step 2: Authentication Check

Before any logic is processed, the backend verifies the user's session. If a request is made to a restricted module (like add_bill.php) and the session is missing or the role is incorrect, the flow

is immediately terminated, and the user is redirected to the login page or sent a "403 Forbidden" status code.

Step 3: Business Logic Execution (The Decision Phase)

Once authenticated, the backend enters the execution phase. For example, if a resident submits a "Complaint":

- The backend retrieves the User_ID from the session.
- It captures the Description from the POST data.
- It applies "Business Rules"—for instance, checking if the resident actually belongs to the flat they are complaining about.

Step 4: Database Transaction

The logic then moves to the **Database Access Layer**. A SQL command is prepared (e.g., INSERT INTO complaints...). The backend binds the parameters and executes the statement. The MySQL engine processes the query and returns a success or failure flag to the PHP script.

Step 5: Logging and Auditing

In advanced implementations, the backend flow includes a logging step. Every time a bill is modified or a visitor is deleted, the backend creates an entry in an internal audit log. This ensures that the society has a trail of who performed which action and at what time.

Step 6: Final Response Encoding

The final stage of the backend flow is the **Response Construction**. The PHP script gathers the result of the database transaction. If successful, it creates an associative array: ['status' => 'success', 'message' => 'Record added']. This array is passed through json_encode() and printed to the output buffer. The flow concludes as the Apache server sends this JSON string back to the client's browser.

Summary of Backend Technologies

Layer	Technology	Role
Server Engine	Apache / PHP 8.x	Handling requests and interpreting scripts.
Security	Bcrypt / PHP Sessions	Password protection and role-based access.
Database Communication	PDO (PHP Data Objects)	Secure, prepared SQL transactions.

Layer	Technology	Role
Data Format	JSON	Standardized communication with the frontend.

5.3 Database Architecture

The database architecture serves as the foundation of the **Society Management System**, acting as the persistent storage layer where all organizational data is structured, managed, and retrieved. Built on the **MySQL** relational database engine, the architecture is designed to handle complex relationships between residents, physical property, and financial transactions while maintaining strict data integrity.

5.3.1 Entities and Attributes

In a relational database, an entity represents a real-world object or concept, while attributes are the specific properties that describe that entity. The following core entities define the scope of the system:

- **User Entity:** Manages the identity of everyone interacting with the system.
 - **Attributes:** User_ID (Primary Key), Full_Name, Email_Address, Password_Hash, and User_Role (Admin/Resident).
- **Flat Entity:** Represents the physical residential units within the society.
 - **Attributes:** Flat_ID (Primary Key), Block_Number, Floor_Level, and Flat_Number.
- **Allotment Entity:** A critical relational entity that links a specific user to a specific flat.
 - **Attributes:** Allotment_ID (Primary Key), User_ID (Foreign Key), Flat_ID (Foreign Key), Move_In_Date, and Status.
- **Bill Entity:** Tracks the financial obligations of each household.
 - **Attributes:** Bill_ID (Primary Key), Flat_ID (Foreign Key), Amount, Billing_Month, and Payment_Status.
- **Visitor Entity:** Logs the movement of external individuals.
 - **Attributes:** Visitor_ID (Primary Key), Visitor_Name, Visit_Reason, In_Time, Out_Time, and Target_Flat_ID (Foreign Key).

5.3.2 Database Design

The design of the database follows a **Relational Schema** optimized through the process of **Normalization**. This ensures that data is stored efficiently without unnecessary redundancy, which is vital for preventing "update anomalies" (where changing a value in one place fails to update it elsewhere).

1. Relational Integrity

The design relies heavily on **Primary Keys** and **Foreign Keys** to create a web of connected data. For example, the Bill table does not store the resident's name; instead, it stores a Flat_ID. When a report is generated, the system performs a **SQL JOIN** operation to pull the resident's name from the User table via the Allotment table. This ensures that if a resident moves out and a new one moves in, the billing history remains tied to the flat, while the person's profile remains distinct.

2. Data Security and Constraints

Security is integrated directly into the database design. The architecture employs **Constraints** to reject invalid data; for instance, a Visitor record cannot be created if the Target_Flat_ID does not exist in the Flat table. Furthermore, sensitive data like passwords are never stored in plain text but as cryptographic hashes within the User attributes, ensuring that even in the event of unauthorized database access, user credentials remain protected.

3. Performance Optimization

To ensure the system remains responsive as the society grows, the design incorporates **Indexing** on frequently searched attributes such as Email_Address and Flat_Number. This allows the backend to locate specific records among thousands of entries in milliseconds, supporting the high-speed search and filter requirements of the frontend dashboard.

Chapter 6: Results & Discussion

6.1 INTRODUCTION

The completion of the **Society Management System (SMS)** represents a transition from conceptual database design to a tangible software solution. The following sections provide a comprehensive breakdown of the results achieved during the 8-week development cycle. This chapter evaluates how the PHP-based logic and MySQL data structures performed under testing, the efficiency of the frontend interface, and a critical discussion of the technical obstacles encountered while building a secure, multi-user environment from scratch.

6.2 RESULTS ACHIEVED

The primary result of this project is a stable, integrated web platform that successfully digitizes the administrative tasks of a housing society. The following milestones represent the core technical outputs of the development phase.

6.2.1 Functional Management and Allotment Logic

The most significant result was the successful creation of a relational allotment engine. Unlike a simple flat list, this module manages the complex relationship between a human resident and a physical property unit.

Relational Integrity:

The backend scripts effectively enforce business rules. For example, the system prevents a "Double Allotment" error. If a flat is currently assigned to one resident, the system rejects any attempt to allot it to another person until the first resident is marked as moved out.

Billing Automation:

A major achievement was the bulk-generation feature. Instead of creating 100 separate invoices, a single SQL procedure can generate monthly maintenance records for every active allotment in the database simultaneously.

6.2.2 Advanced Information Retrieval

By integrating the jQuery DataTables plugin with MySQL queries, the system achieved high-speed data searching that feels instantaneous to the user.

Global Search Functionality:

On the Visitor and Billing pages, users can filter through hundreds of historical records using a single search bar. The results are filtered on the client side, which drastically reduces the load on the local XAMPP server.

Role-Based Access Control (RBAC):

The system successfully implemented a security gate. During testing, it was verified that a user logged in as a resident is automatically blocked if they attempt to access admin-only pages like user management or system settings.

6.2.3 Responsive Frontend Performance

The system was designed to be accessible for security guards at the gate and managers in the office.

Adaptive Grid Layout:

Using Bootstrap 5, the dashboard cards and data tables were tested across various screen sizes. The sidebar remains collapsible and the data columns remain readable on mobile browsers, which is essential for logging visitors at the society entrance.

Consistent UI Components:

The use of the SB Admin Theme ensured that the action buttons, such as edit and delete, occupy the same location across all modules, making the system easy to learn for new administrators.

6.2.4 Database and Security Integration

The MySQL database proved to be a reliable backbone for the entire system.

Data Security:

The implementation of Bcrypt hashing through the PHP password_hash function was a critical success. In the database, passwords appear as 60-character random strings, ensuring that resident credentials remain protected even if the database is exported.

Prepared Statements:

Every query in the system uses PDO Prepared Statements. This choice successfully stopped SQL Injection risks during security testing, as the database treats all user input as literal text rather than executable commands.

6.2.5 Comprehensive Testing Outcomes

Before the project was finalized, it underwent a rigorous testing process:

Unit Testing:

Individual components, like the function for calculating total dues, were tested with different number formats to ensure mathematical accuracy.

Integration Testing:

The flow from the frontend form to the PHP logic and finally to the MySQL database commit was verified to be fully reliable.

Stress Testing:

The system was loaded with 500 dummy resident records to ensure that the search and pagination features did not slow down the browser.

6.3 DISCUSSION

Building this system provided a practical look at the challenges of full-stack engineering. While the code is stable, the journey involved several breakthroughs and troubleshooting sessions.

6.3.1 Technical Learnings

The project helped bridge the gap between simple scripts and a connected software ecosystem.

The Controller Pattern:

Learning to use a centralized `action.php` file was a turning point. Initially, the plan was to have a separate file for every button, but that became impossible to manage. Consolidating the logic into one controller made the system much easier to fix and expand.

Asynchronous UX:

Before this project, every button click usually meant a full page refresh. Implementing AJAX showed how background data transfers can make a web app feel like a fast desktop application.

6.3.2 Technical Challenges and Solutions

Database Connectivity:

Several hurdles required deep-diving into documentation to find a solution:

- **The SQL Join Problem:** Pulling a resident's name, their flat number, and their current bill status required joining four different tables. Writing this query without creating duplicate rows was the hardest SQL task. The solution involved using LEFT JOINS with specific constraints to keep the data accurate.
- **Handling Visitor Timestamps:** Managing out-times for visitors who had not yet left was tricky. The database had to be configured to allow NULL values, and the PHP code had to be written to display "Still Inside" instead of an empty box if no exit time was recorded.

6.4 REFLECTIONS

Beyond coding and testing, the project had a more **personal and reflective side**.

The process of developing the Society Management System served as a significant milestone in transitioning from academic theory to practical application. Moving beyond isolated coding exercises to build a cohesive, multi-module application required a fundamental shift in mindset from simply making code work to making code sustainable and secure.

One of the most valuable lessons involved the discipline of planning. At the start of the project, there was a strong urge to begin with the visual design of the dashboard. However, the time spent on the Data Dictionary and ER Diagrams proved to be the most critical investment. A well-structured database made the PHP logic straightforward to implement, whereas any small oversight in the table relationships caused significant bugs in the frontend. This experience reinforced the principle that a solid backend is the foundation of any reliable application.

Furthermore, the project highlighted the complexities of full-stack development. Balancing the frontend interactivity with backend security was a challenging learning curve. Realizing that a single user action, such as adding a visitor, required synchronized changes across the HTML form, the jQuery AJAX script, and the PHP controller was an eye-opening experience. It

increased the understanding of modular programming and the importance of keeping logic organized.

Finally, the project instilled a permanent security-first mindset. Implementing Bcrypt hashing and PDO Prepared Statements transformed security from a theoretical concept into a practical responsibility. It became clear that as a developer, the safety of user data is a primary obligation, and shortcuts in security are never acceptable.

6.5 KEY OBSERVATIONS

During the final testing and evaluation phase, several technical and behavioral observations were made regarding the system's performance and usability.

Database Integrity is Mandatory:

It was observed that data can easily become orphaned or inconsistent without proper constraints. For instance, if a flat is deleted while billing records still exist, the system generates errors during financial reporting. This observation led to the realization that Foreign Key Constraints and specific delete rules are essential for maintaining a healthy database over time.

User Preference for Simplicity:

Observations during peer testing showed that users are often overwhelmed by dense information. The most successful feature was the DataTables live search. Users preferred a clean interface where they could find records by typing a few characters rather than navigating through complex filter menus.

The Efficiency of Local Stacks:

For a management system of this scale, the XAMPP environment proved to be remarkably robust. It was observed that a PHP-MySQL application can easily handle the data requirements for a medium-sized society of several hundred units without performance lag, provided that SQL queries are properly indexed.

The Necessity of Double Validation:

A critical observation was that user error is inevitable. While HTML5 validation helps, it is easily bypassed. Therefore, the backend must always perform its own independent validation to act as the final judge of what data is permitted to enter the database.

CHAPTER 7: CONCLUSION & FUTURE SCOPE

The development of the **Society Management System (SMS)** serves as a comprehensive response to the operational inefficiencies found in traditional, manual residential administration. By replacing physical registers and fragmented spreadsheets with a centralized digital platform, the project successfully streamlines the core functions of community management. This chapter summarizes the project's outcomes, evaluates the technical achievements against the initial objectives, and identifies the path for future enhancements.

7.1 CONCLUSION

The successful implementation of the Society Management System demonstrates the effectiveness of utilizing a **3-Tier Architecture** to manage complex residential data. By isolating the user interface from the server-side logic and the database storage, the system ensures a high level of stability and performance. The transition from paper-based record-keeping to a robust **MySQL** environment has effectively eliminated the risks of data degradation, unauthorized physical access, and human calculation errors.

Throughout the development process, the primary focus remained on creating a reliable tool that could handle the daily demands of a housing society without requiring high-end hardware. The choice of **Procedural PHP** and the **WAMP stack** provided a direct and efficient path to achieving these goals. The final product stands as a secure, functional, and user-centric platform that centralizes visitor logs, resident records, and financial transactions. Ultimately, the system fulfills the requirement for a modern administrative solution that promotes transparency and operational speed within a residential community.

7.2 ACHIEVEMENTS

The project reached several significant technical milestones that validate the chosen development strategy. These achievements reflect a commitment to both security and user experience:

Implementation of Robust Security Protocols:

A major achievement is the integration of high-level security features within the backend logic. The use of **Bcrypt hashing** ensures that user passwords are never stored in plain text, while the adoption of **PDO Prepared Statements** effectively neutralizes the threat of SQL injection attacks. These measures provide a professional level of data protection.

Responsive and Adaptive Design:

By leveraging the **Bootstrap 5 framework**, the system achieved full responsiveness. The dashboard automatically adjusts its layout to maintain usability on a variety of devices, ensuring that security personnel at gates can use tablets or smartphones with the same efficiency as an administrator using a desktop computer in the office.

Seamless Asynchronous Interactivity:

The integration of **AJAX and JSON** transformed the application from a series of static pages into a dynamic ecosystem. This technical choice allows the system to save data and fetch records in the background, providing a fluid user experience that eliminates the latency associated with full page refreshes.

Centralized Financial Transparency:

The billing module successfully automates the generation and tracking of maintenance dues. Through the use of color-coded status indicators and automated list generation, the system provides an immediate overview of the society's financial health, reducing the time required for monthly auditing by a significant margin.

Advanced Data Management:

The incorporation of the **DataTables plugin** allows for the management of large datasets with ease. The "live search" and instant sorting features empower administrators to retrieve specific visitor or resident records among hundreds of entries in milliseconds.

7.3 LIMITATIONS

While the current version of the Society Management System is fully operational, certain technical and functional limitations exist that define the boundaries of the current scope:

Local Network Constraint:

The application currently operates within a **local XAMPP environment**. While this is effective for internal society use, the system is not yet deployed on a public cloud server, which limits remote access from outside the society's local network or Wi-Fi range.

Absence of Automated Notifications:

The system currently relies on users logging into the dashboard to check for updates. There is no integrated **SMS or Email API** to send automated alerts to residents regarding new maintenance bills, emergency announcements, or the arrival of visitors.

Manual Payment Verification:

Due to the complexities of integrating financial APIs, a live **Payment Gateway** is not included in this version. Consequently, all payment status updates require manual intervention by the administrator after verifying bank transfers or cash receipts offline.

Limited Multimedia and File Support:

The database is optimized for text and numerical data. The system does not currently support the uploading of high-resolution images or documents, such as resident identity proofs or photos for maintenance complaint logs.

7.4 FUTURE SCOPE

The Society Management System is designed with a modular architecture that allows for significant scaling and the addition of advanced features in future iterations:

Cloud Hosting and Mobile Application:

The next logical phase is the deployment of the system to a secure cloud hosting platform. This would be followed by the development of dedicated **Android and iOS applications**, enabling push notifications for residents and real-time alerts for security personnel.

IoT and Hardware Integration:

Future versions could integrate with physical hardware. For instance, a **QR Code-based Visitor Pass** system could be developed where guests receive a code on their mobile devices, which is then scanned at the security gate to automatically log their entry and exit in the MySQL database.

Live Payment Gateway Integration:

The inclusion of a secure payment API (such as Razorpay or Stripe) would allow residents to pay maintenance dues directly through the portal. This would enable automated receipt generation and instant status updates in the database, removing the need for manual administrative work.

Automated Communication Engine:

By integrating third-party services like **Twilio or SendGrid**, the system could be upgraded to send automated reminders for unpaid bills, meeting notices, and urgent security updates directly to residents' mobile phones.

Enhanced Security with Facial Recognition:

Advanced versions of the system could utilize AI-driven facial recognition at the entry gates to identify residents and pre-authorized visitors, further strengthening the society's security protocols.

Chapter 8: Bibliography

8.1 CORE BACKEND AND LOGIC RESOURCES

The server-side logic of the system was built using Procedural PHP and the WAMP stack. The following references provided the necessary framework for handling session management, data processing, and server-side validation.

The PHP Group (2025). PHP Manual. The official documentation served as the primary reference for understanding core functions, array handling, and the implementation of PHP Data Objects (PDO). It was specifically consulted for the syntax of `password_hash()` and `password_verify()` to ensure the encryption module met modern cryptographic standards.

Ullman, L. (2017). PHP and MySQL for Dynamic Web Sites: Visual QuickPro Guide. This text provided the foundational logic for the centralized controller architecture (`action.php`). The concepts of handling form data and managing state across multiple pages were derived from the methodologies outlined in this guide.

Nixon, R. (2021). Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5. This comprehensive guide assisted in the integration of asynchronous requests. The transition from standard form submissions to the AJAX-based model used in the visitor and billing modules was informed by the examples provided in this resource.

8.2 DATABASE MANAGEMENT AND DESIGN

Ensuring that the Society Management System maintained data integrity required a deep understanding of relational database design and normalization.

Date, C. J. (2003). An Introduction to Database Systems. This classic academic text was used to establish the rules for Third Normal Form (3NF). The design of the allotments table as a linking entity between users and flats was a direct application of the relational principles discussed in this book.

Oracle Corporation (2025). MySQL 8.0 Reference Manual. The official MySQL documentation was utilized to optimize SQL queries. It was particularly useful when writing complex LEFT JOIN operations to fetch resident names and flat numbers for the monthly billing reports.

Beighley, L., & Morrison, M. (2006). Head First SQL. This resource provided a practical approach to understanding foreign key constraints and referential integrity. These concepts were essential in preventing orphaned records in the billing and visitor logs when a resident or flat is deleted from the system.

8.3 FRONTEND FRAMEWORKS AND UI/UX

The visual presentation and responsiveness of the dashboard were achieved through the application of the Bootstrap framework and custom Cascading Style Sheets (CSS).

Otto, M., & Thornton, J. (2025). Bootstrap 5 Official Documentation. The grid system and utility classes used to create the mobile-friendly dashboard were sourced from this documentation. The implementation of cards, navbars, and modal forms followed the standardized accessibility guidelines provided by the framework developers.

Duckett, J. (2011). HTML & CSS: Design and Build Websites. This reference guided the customization of the SB Admin theme. It was used to write the specific CSS overrides for the color-coded status badges (e.g., success for paid bills and danger for unpaid dues).

jQuery Foundation (2025). jQuery API Documentation. The logic for intercepting form submissions and processing them in the background was based on the jQuery AJAX API. This reference was vital for implementing the real-time search and delete functionality within the resident management lists.

8.4 SECURITY AND ARCHITECTURAL BEST PRACTICES

Security was a primary objective of the project. The following resources were consulted to protect the system against common web vulnerabilities.

OWASP Foundation (2021). OWASP Top Ten Project. The guidelines for preventing SQL Injection and Broken Access Control were strictly followed. This research led to the decision to utilize PDO Prepared Statements instead of traditional MySQLi queries.

Spaanjaars, I. (2014). Beginning ASP.NET 4.5.1: in C# and VB (Wrox Programmer to Programmer). Although focused on a different language, the architectural principles of 3-Tier Systems discussed in this text were applied to the PHP environment to ensure a clean separation between the presentation, logic, and data layers.

W3Schools Online Web Tutorials (2025). Web Security Tutorials. These modules provided practical examples of session hijacking prevention. The use of `session_start()` and role-based redirection logic was refined based on the best practices outlined in these tutorials.

8.5 SOFTWARE ENGINEERING METHODOLOGIES

The systematic approach to the development lifecycle was informed by established software engineering models.

Sommerville, I. (2015). Software Engineering. This textbook provided the framework for the Waterfall model used during the project's development. The structured stages of requirement analysis, design, implementation, and testing described by Sommerville were used to keep the project on schedule and within its defined scope.

Larman, C. (2004). Applying UML and Patterns. This resource was consulted during the creation of the System Flowcharts and Entity-Relationship Diagrams (ERD). The use of standard UML notation ensured that the architectural diagrams were technically accurate and easy to interpret for the purpose of the project documentation.

The bibliography represents a blend of official technical manuals, pedagogical textbooks, and professional security guidelines. These sources collectively ensured that the Society Management System was built on a foundation of proven methodologies and modern web standards.
